Universidad Técnica Estatal de Quevedo

Facultad de Ciencias de la Ingeniería

Carrera: Ingeniería de Software

Asignatura: Aplicaciones Distribuidas

Código de la actividad: TA-IND-01

Informe Académico Individual:
Análisis crítico de CockroachDB frente a Apache Cassandra

Estudiante: Toaquiza Zambrano Belinda Betzabeth

Paralelo: A

Fecha: 13 de noviembre de 2026

Video de presentación: https://youtu.be/jjIqFWVfyhc

Resumen

Este informe analiza críticamente a CockroachDB frente a Apache Cassandra a partir de los conceptos de la Unidad 1: transparencias de distribución, modelos de comunicación, teorema CAP y PACELC, y tolerancia a fallos. El estudio combina una revisión documental de documentación técnica y literatura académica con dos experimentos propios sobre clústeres de tres nodos ejecutados con Docker. El primero detiene uno y dos nodos para observar lecturas y escrituras; el segundo aísla un nodo de la red para construir una partición 2+1. Los resultados muestran que CockroachDB conserva la coherencia mediante quórum Raft y deja de confirmar operaciones al perder la mayoría, mientras que Apache Cassandra mantiene la disponibilidad según el nivel de consistencia elegido: `ONE` operó con dos nodos caídos y en ambos lados de la partición, y `QUORUM` y `ALL` fallaron cuando faltaron réplicas. El análisis concluye que la diferencia principal no está en la topología, sino en dónde se fija el compromiso entre consistencia y disponibilidad, y que la elección depende del requisito de la aplicación. Se reconoce como limitación que el ensayo no midió latencia ni escrituras concurrentes sobre una misma clave.

Palabras clave: sistemas distribuidos; consistencia; disponibilidad; tolerancia a fallos; CockroachDB; Apache Cassandra.

Abstract

This report critically analyzes CockroachDB against Apache Cassandra using the concepts of Unit 1: distribution transparency, communication models, the CAP and PACELC theorems, and fault tolerance. The study combines a documentary review of technical documentation and academic literature with two experiments on three-node clusters running on Docker. The first stops one and two nodes to observe reads and writes; the second isolates a node from the network to build a 2+1 partition. The results show that CockroachDB preserves consistency through Raft quorum and stops confirming operations when it loses the majority, whereas Apache Cassandra maintains availability according to the chosen consistency level: `ONE` operated with two nodes down and on both sides of the partition, while `QUORUM` and `ALL` failed when replicas were missing. The analysis concludes that the main difference lies not in the topology but in where the trade-off between consistency and availability is fixed, and that the choice depends on the application requirement. A limitation is acknowledged: the experiment did not measure latency or concurrent writes to the same key.

Keywords: distributed systems; consistency; availability; fault tolerance; CockroachDB; Apache Cassandra.

1. Introducción

Las aplicaciones actuales rara vez dependen de un único servidor. Los servicios de banca, comercio, mensajería y trámites públicos atienden a millones de usuarios y necesitan conservar los datos aunque un equipo falle, una red se corte o la demanda crezca de un día para otro. Para responder a esa exigencia surgieron las bases de datos distribuidas, que reparten la información entre varios nodos y coordinan las operaciones entre ellos. Esa coordinación introduce un problema que la Unidad 1 estudia de forma explícita: cuando la red falla, el sistema debe elegir entre mantener la coherencia de lo que confirma o seguir respondiendo con la información disponible. El teorema CAP y el modelo PACELC describen ese compromiso, y las decisiones de diseño de cada motor lo reflejan en su arquitectura [1], [2].

CockroachDB representa la familia de bases de datos SQL distribuidas que conservan las transacciones y el lenguaje relacional. Su arquitectura organiza los datos en rangos replicados y utiliza consenso Raft para confirmar cada escritura con la mayoría de las réplicas votantes; además, ofrece funciones para desplegar tablas en varias regiones y atender lecturas en réplicas de seguidor [3], [4]. Apache Cassandra, en cambio, proviene del modelo Dynamo y de las bases de datos de columnas anchas: los datos se distribuyen por tokens, cada clave tiene un factor de replicación y el cliente decide cuántas réplicas deben responder mediante niveles como `ONE`, `QUORUM` y `ALL` [2]. Su adopción en servicios de gran escala, como el almacenamiento de mensajes de Discord, muestra que el modelo sostiene volúmenes muy altos, aunque también expone el costo operativo de mantenerlo [5].

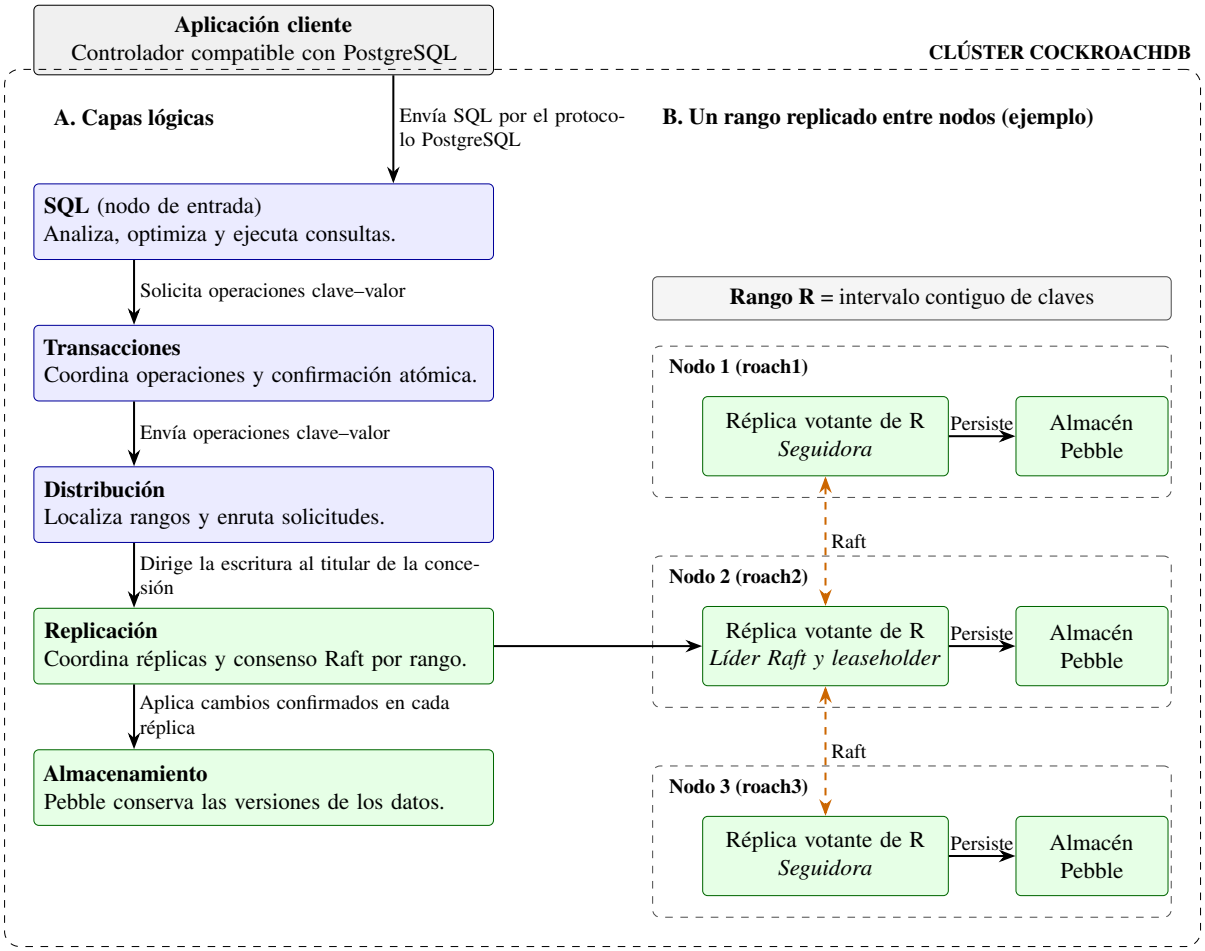
La elección entre ambos motores suele resolverse con afirmaciones generales, del tipo «uno es más consistente» o «el otro es más disponible», sin analizar qué operación se ejecuta, con cuántas réplicas y bajo qué falla. El problema que aborda este informe es precisamente ese: determinar qué garantías ofrece cada sistema en condiciones concretas y qué costo asume la aplicación por ellas. Por eso el objetivo general es analizar críticamente a CockroachDB frente a Apache Cassandra a partir de los conceptos de la Unidad 1, la documentación técnica y una ejecución real. Los objetivos específicos son describir la arquitectura de CockroachDB, contrastar ambos motores con los criterios de transparencia, comunicación, CAP/PACELC y tolerancia a fallos, diseñar y ejecutar dos experimentos que pongan a prueba la caída de nodos y la partición de red, y formular una postura argumentada sobre los contextos en los que cada uno resulta adecuado.

La metodología combina dos fuentes de evidencia. La primera es documental: se revisaron la documentación oficial de ambos proyectos, artículos académicos sobre consistencia y rendimiento en bases NoSQL, y casos de operación a gran escala [5], [6], [7]. La segunda es experimental: se levantaron dos clústeres de tres nodos con Docker sobre una misma máquina, se detuvieron nodos y se aisló uno de la red para observar lecturas y escrituras en cada configuración. Los comandos, las horas y las salidas completas están en `experimento/comandos.md`, y las capturas de evidencia en la carpeta `Capturas`. Cada paso se registró antes y después de ejecutarlo, con la terminal identificable y la hora visible, y las capturas se contrastaron con las salidas de la consola. El alcance se limita a esas condiciones: una topología, unas versiones y unas operaciones, sin mediciones de latencia ni escrituras concurrentes sobre una misma clave.

El documento se organiza en ocho secciones. La segunda describe la arquitectura de CockroachDB; la tercera aplica los conceptos de la Unidad 1; la cuarta presenta los dos experimentos con su procedimiento, resultados y conclusiones; la quinta relaciona lo observado con dos servicios de uso cotidiano; la sexta compara ambos motores con cinco criterios; la séptima expone la postura crítica con fortalezas, limitaciones y contextos de uso; y la octava recoge las conclusiones. Los anexos conservan el esquema inicial y las capturas de lectura de las fuentes. Los archivos de composición y el registro de comandos quedan en el repositorio para reproducir el ensayo.

2. Arquitectura del sistema

La aplicación cliente envía consultas a un nodo de CockroachDB mediante un controlador compatible con PostgreSQL. La capa SQL analiza y ejecuta las consultas, mientras que la capa de transacciones coordina las operaciones de lectura y escritura. La capa de distribución localiza los rangos de claves y dirige cada operación a las réplicas correspondientes, como se describe en la arquitectura del sistema [3]. En el ejemplo del diagrama, la réplica que coordina la escritura es el líder Raft y se comunica con las otras dos réplicas. La escritura se confirma al alcanzar el quórum requerido, es decir, al replicarse en al menos dos de los tres nodos [1]. Finalmente, cada nodo conserva los datos en su almacenamiento local Pebble [8].



Leyenda: flecha continua = solicitud o aplicación local; flecha discontinua naranja = comunicación Raft; cajas azules = procesamiento y enrutamiento; cajas verdes = replicación y almacenamiento. Los roles de Raft se asignan por rango; el líder mostrado es un ejemplo.

Figura 1: Arquitectura lógica de CockroachDB y réplica de un rango en el clúster de tres nodos del experimento (elaboración propia).

El esquema preliminar mostraba las cinco capas de CockroachDB y un ejemplo de un rango replicado en tres nodos. La versión final amplió esa vista al detallar la función de cada capa y el recorrido de una solicitud desde la aplicación cliente hasta el almacenamiento. También distinguió la réplica líder de las seguidoras y mostró cómo se comunican mediante Raft, cuántas réplicas se necesitan para confirmar una escritura y cómo se guardan los cambios en Pebble en cada nodo.

3. Análisis con los conceptos de la Unidad 1

3.1. Transparencias de distribución

La transparencia de distribución describe cuánto puede usar una aplicación los datos sin tener que conocer su ubicación física, el número de réplicas o los cambios internos del clúster. En ambos motores hay transparencias parciales: la interfaz simplifica el acceso, pero las decisiones sobre consistencia, disponibilidad y recuperación siguen siendo visibles para quien opera o programa el sistema.

Acceso. CockroachDB ofrece una interfaz SQL compatible con el protocolo de red de PostgreSQL; traduce las consultas en solicitudes sobre su almacenamiento distribuido. Cassandra expone CQL mediante su protocolo binario nativo. En los dos casos, el usuario trabaja con una interfaz lógica sin enviar manualmente cada consulta a todas las réplicas [3], [9], [10].

Ubicación. Ambos sistemas ocultan en gran medida dónde reside cada dato. CockroachDB divide el espacio de claves en rangos y distribuye el encaminamiento entre los nodos. Cassandra calcula el token de la clave de partición y entrega la solicitud a las réplicas pertinentes; cualquier nodo puede actuar como coordinador. La ubicación física no forma parte de la consulta normal, aunque sí puede ser relevante para la configuración y la observabilidad [2], [3].

Migración. La ubicación lógica también permanece separada de la consulta cuando cambian las réplicas. CockroachDB puede reequilibrar rangos entre nodos; Cassandra distribuye tokens en el anillo y permite añadir, mover o retirar nodos. Esta transparencia es operativa, no absoluta: el cambio de topología requiere configuración y puede involucrar movimiento de datos, carga de red y tareas de mantenimiento [1], [2].

Reubicación. La ocultación de una transferencia mientras hay carga es parcial. CockroachDB describe la transferencia de instantáneas y la puesta al día de réplicas durante el rebalanceo. En Cassandra, los cambios del anillo y la reparación pueden transferir datos entre nodos. Las fuentes describen estos mecanismos, pero no garantizan que toda consulta en curso termine sin demora o reintento; esa continuidad tampoco se midió en esta práctica [1], [2], [11].

Replicación. CockroachDB administra réplicas de rangos mediante Raft y el cliente no selecciona los nodos que deben confirmar una escritura. Cassandra replica cada partición según la estrategia del espacio de claves, pero expone al cliente los niveles de consistencia (`ONE`, `QUORUM`, `ALL`, entre otros), que fijan cuántas respuestas espera el coordinador. Por eso Cassandra automatiza la copia de datos, aunque deja visible una parte mayor de la política de lectura y escritura [1], [2].

Concurrencia. CockroachDB aplica aislamiento `SERIALIZABLE` por defecto: el efecto de transacciones concurrentes debe corresponder a algún orden serial. La contención puede provocar reintentos y, si no se resuelve automáticamente, un error visible para la aplicación. Cassandra resuelve mutaciones ordinarias concurrentes por marcas de tiempo y regla de último escrito; no ofrece por ello una transacción serializable global entre particiones. Para operaciones condicionales de una partición ofrece transacciones ligeras basadas en Paxos. Así, en los dos motores la aplicación puede tener que manejar contención o elegir una garantía concreta [2], [6], [9].

Fallos. La abstracción también es parcial. CockroachDB puede seguir atendiendo operaciones que conservan mayoría de votantes para el rango, pero puede rechazar una lectura actual o una escritura si

falta quórum; también puede perderse la conexión al nodo usado por el cliente. Cassandra deja que el coordinador atienda según el nivel elegido, por lo que una operación con `ONE` puede responder cuando niveles mayores no alcanzan sus réplicas requeridas. Los errores de disponibilidad, los tiempos de espera y los reintentos no quedan ocultos por completo [1], [2].

3.2. CAP y PACELC

CAP y PACELC describen compromisos bajo supuestos concretos, no una etiqueta independiente de la operación. En lo que sigue, la clasificación se limita a lecturas actuales y escrituras ordinarias sobre datos con tres réplicas; las conclusiones empíricas se restringen a las operaciones registradas en los experimentos [1], [2], [6].

Durante una partición de red, CockroachDB prioriza mantener la consistencia de las operaciones que confirma: Raft exige mayoría de las réplicas del rango. En una disposición de tres réplicas, el lado con dos puede avanzar y el lado con una no puede confirmar una escritura que requiere quórum. Por ello, para estas operaciones CockroachDB presenta el perfil CP: tolera la partición, pero sacrifica disponibilidad en la minoría. En Cassandra el compromiso depende del nivel por operación. La documentación describe su orientación AP; en la práctica, `ONE` puede responder en cada lado si cada uno alcanza una réplica, mientras que `QUORUM` solo alcanza el mínimo en la mayoría y `ALL` requiere las tres. Con niveles más exigentes, Cassandra también sacrifica disponibilidad en el lado que no reúne las respuestas [1], [2], [6].

Los ensayos concuerdan con ese contraste, dentro del alcance de las salidas observadas. Las salidas propias constan en `experimento/comandos.md`, apartados de E1 y E2. En E1, CockroachDB leyó y escribió con un nodo detenido; al detener dos, la consulta devolvió `replica unavailable`. En E2, el lado mayoritario confirmó la fila 31 y la lectura actual desde `roach3` aislado falló con `lost quorum`; el `INSERT` de la fila 32 quedó sin respuesta visible, así que no se afirma si se confirmó. En Cassandra, la partición 2+1 mostró que `ONE` completó lecturas y escrituras en ambos lados; `QUORUM` solo en el lado con dos réplicas; y `ALL` falló en ese lado y no se probó en el aislado. Tras reconectar, `ALL` devolvió las filas 21, 23 y 24. Como se usaron claves distintas, la prueba no demuestra cómo se resolverían escrituras concurrentes sobre la misma clave [1], [2].

PACELC añade la pregunta del comportamiento sin partición: consistencia o menor latencia. Para operaciones actuales, CockroachDB mantiene coordinación de consenso y confirmación por mayoría, lo que puede añadir viajes de red; ofrece lecturas históricas de seguidores cuando se acepta menor frescura. El trabajo de VanBenschoten et al. estudia configuraciones de localidad y lecturas locales consistentes en despliegues multirregión; sus resultados explican una optimización posible, pero no predicen la latencia de este clúster local. La inferencia para las operaciones actuales de esta práctica es un perfil PC/EC: ante partición conserva consistencia en la mayoría y, aun sin partición, conserva consistencia mediante coordinación [1], [4], [12].

Cassandra permite escoger cuánto espera el coordinador. `ONE` exige menos respuestas y puede favorecer menor espera; `QUORUM` y `ALL` exigen más confirmaciones y restringen las operaciones que pueden responder. Por ello, su perfil sin partición no es único: el nivel elegido mueve el compromiso hacia latencia o consistencia. Ferreira et al. compararon niveles de consistencia en Cassandra y otros motores en despliegues de nube, y hallaron diferencias de rendimiento asociadas a la configuración; da Silva y Lima evaluaron sistemas en un clúster de placas de bajo consumo. Nejati SharifAldin y Nayebi Pour compararon los niveles nativos con un middleware causal externo. Estos estudios usan cargas y topologías distintas y respaldan el compromiso cualitativo, no una predicción numérica para Docker. En la práctica no se midió latencia; por tanto, la clasificación PL/EL para `ONE` y PC/EC para niveles de quórum se presenta como inferencia documental y no como resultado de rendimiento del experimento [2], [7], [13], [14].

3.3. Modelos de comunicación

CockroachDB combina el modelo cliente-servidor hacia la aplicación con coordinación distribuida entre nodos. El cliente envía SQL por el protocolo PostgreSQL a un nodo; internamente, los nodos participantes coordinan transacciones y replicación por rangos mediante Raft. La llamada del cliente es síncrona en el sentido de que espera éxito o error de la operación; la escritura no se confirma antes de obtener el quórum requerido. La replicación, el rebalanceo y la actualización de réplicas ocurren mediante intercambios de red entre nodos. El artículo de Kettaneh et al. describe el mecanismo Leader Leases y su detector distribuido de fallos, mientras el documento de multirregión analiza el efecto de la localidad en las transacciones. Esta descripción es arquitectónica; las capturas de la práctica no exponen los mensajes internos ni su temporización [1], [3], [4], [9], [15].

Cassandra también recibe solicitudes de clientes en un modelo cliente-servidor, pero su clúster opera entre pares: un nodo accesible puede coordinar cada petición y distribuirla a las réplicas. El protocolo nativo transporta mensajes CQL desde el cliente; entre nodos se usa mensajería internodo, y gossip intercambia información de membresía y estado. La respuesta al cliente es síncrona respecto al nivel CQL seleccionado: el coordinador espera el número requerido de respuestas. En paralelo, gossip, hinted handoff y reparación participan en la propagación o convergencia posterior, por lo que el sistema combina comunicación de petición-respuesta con trabajo de fondo asíncrono [2], [10], [11], [16], [17].

3.4. Tolerancia a fallos

Con tres votantes para un rango, CockroachDB puede tolerar la pérdida de uno mientras los otros dos se comunican. Si se pierde la mayoría, no confirma nuevas escrituras de ese rango; otras partes del clúster pueden seguir funcionando si conservan sus propios quórums. Raft mantiene el registro replicado y el sistema puede transferir liderazgo y concesiones después de detectar que un nodo no está disponible. El artículo de Kettaneh et al. describe Leader Leases con una detección de vitalidad entre nodos. Como la práctica no inspeccionó métricas internas ni el algoritmo exacto de detección activo, los resultados externos no permiten verificar ese mecanismo directamente [1], [15].

En E1, con `roach2` detenido, CockroachDB mantuvo lectura y escritura; con `roach2` y `roach3` detenidos, la consulta no alcanzó quórum. Al iniciar los contenedores, el servicio SQL volvió a responder y las filas confirmadas siguieron disponibles. Sin embargo, `node status` mostró un estado anómalo de `roach1` hasta que se reinició; la causa de ese estado no quedó establecida. En E2, con `roach3` desconectado de la red pero aún encendido, la escritura desde la mayoría funcionó y la lectura actual desde el nodo aislado devolvió `lost quorum`. Ese escenario comprueba una partición de comunicación, distinta de detener el proceso [1].

Cassandra detecta disponibilidad de pares a partir de heartbeats y gossip. Si una réplica no está disponible, el coordinador puede conservar una mutación pendiente mediante hinted handoff; ese mecanismo es de mejor esfuerzo. La reparación anti-entropía compara rangos con árboles de Merkle y transmite diferencias, y requiere ejecución operativa con `nodetool repair`. En E1, con RF=3, `ONE` y `QUORUM` funcionaron tras detener un nodo, mientras `ALL` falló; con dos nodos detenidos, solo `ONE` completó la lectura y la escritura comprobada. Tras iniciar nodos, el estado llegó a tres `UN` y `ALL` leyó las filas confirmadas. En E2, los tres nodos volvieron a `UN`, pero las consultas posteriores no verifican por sí mismas si se usaron hints o repair; el mecanismo exacto de convergencia quedó sin medir [2], [6], [11].

Como caso operativo real, Discord describió que su clúster Cassandra de mensajes llegó a 177 nodos y trillones de mensajes, con latencia impredecible, alta carga de mantenimiento y degradación en cascada asociada a particiones calientes y consultas con consistencia de quórum. La publicación relata una degradación de rendimiento y operación, no una caída total del servicio ni una falla de consenso. El caso muestra que replicación y disponibilidad no evitan problemas producidos por patrones de acceso sesgados y carga; no permite trasladar sus magnitudes a esta práctica [5]. El análisis de latencia permanece

cualitativo porque en los experimentos no se registraron mediciones temporales comparables.

4. Experimentación propia

Esta sección contrasta con una ejecución real las diferencias que el análisis documental atribuyó a cada motor. Se realizaron dos experimentos sobre la misma topología de tres nodos: el primero detiene nodos y observa qué ocurre con las lecturas y las escrituras; el segundo aísla un nodo de la red para construir una partición 2+1. El registro completo de comandos, horas y salidas que no se reproducen aquí está en `experimento/comandos.md` del repositorio.

4.1. Entorno y justificación

Ambos sistemas se ejecutaron con Docker sobre Windows, en PowerShell y en una sola máquina (`DESKTOP-OCH81AB`) con Docker 29.9.3 (build 5457199). Se usaron los archivos de composición de `experimento/cockroachdb/` y `experimento/cassandra/`. CockroachDB correspondió a la versión 26.4.0 (CCL), con la imagen `cockroachdb/cockroach:v26.4.0` y tres nodos (`roach1`, `roach2` y `roach3`) que se comunican por TLS con certificados generados para el ejercicio y guardados en el volumen `crdb-certs`. Apache Cassandra correspondió a la versión 5.0.9, con la imagen `cassandra:5.0` y tres nodos (`cass1`, `cass2` y `cass3`) en el centro de datos `dc1`; el espacio de claves `prueba` usó `NetworkTopologyStrategy` con factor de replicación 3.

La elección de Docker se justifica por tres razones prácticas. Primera, reproduce la misma topología en cada ejecución a partir de los archivos de composición, sin instalar los motores en el sistema anfitrión. Segunda, separa los nodos en una red virtual propia, lo que permitió construir la partición del segundo experimento con `docker network disconnect` sin apagar el proceso. Tercera, deja las salidas de cada nodo disponibles mediante `docker exec`, de modo que la evidencia se toma del sistema en ejecución. La contrapartida es que los tres nodos comparten un solo equipo físico: las caídas y el aislamiento ocurren entre contenedores y no entre máquinas, por lo que la prueba no reproduce fallas de hardware ni latencias de una red real.

Cada rango de CockroachDB se replica en tres réplicas votantes y las escrituras se confirman al alcanzar la mayoría, que en esta topología son dos de tres réplicas [1]. En Apache Cassandra, el factor de replicación 3 indica cuántas copias guarda el clúster, mientras que el nivel de consistencia de cada operación fija cuántas respuestas espera el coordinador; por eso `ONE`, `QUORUM` y `ALL` exigen una, dos y tres respuestas respectivamente [2], [6].



Figura 2: Inicialización del clúster de CockroachDB y verificación de los tres nodos con `node status` (13 de noviembre de 2026).

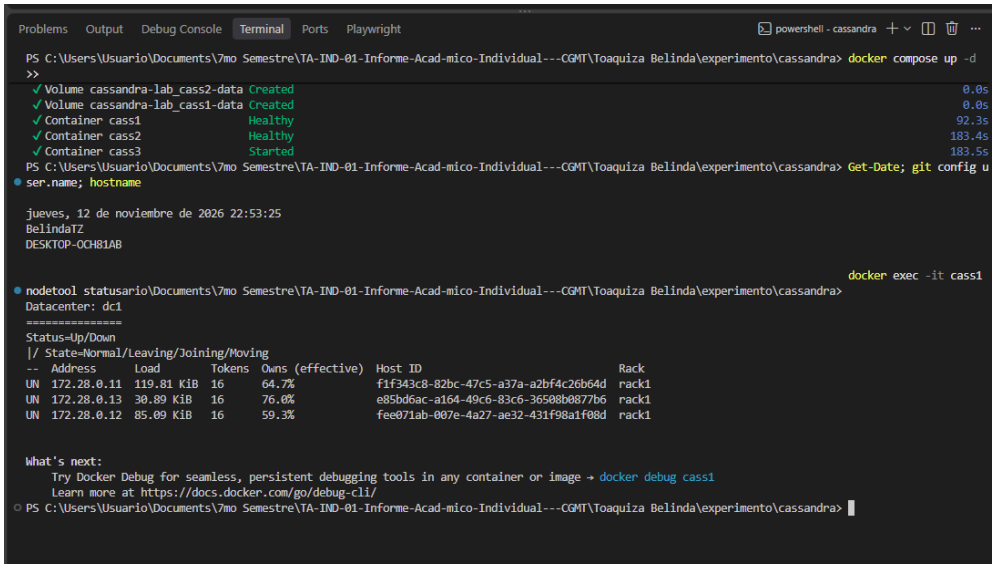


Figura 3: Clúster de Apache Cassandra con los tres nodos en estado `UN` según `nodetool status`, que confirma la disponibilidad del anillo [16].

Listing 1: Arranque de ambos clústeres (extracto de `experimento/comandos.md`).

```
# CockroachDB (desde experimento/cockroachdb)
docker compose up -d
docker compose ps
docker exec -it roach1 cockroach init --certs-dir=/certs --host=roach1:26257
docker exec -it roach1 cockroach node status --certs-dir=/certs --host=roach1:26257

# Cassandra (desde experimento/cassandra)
```

```
docker compose up -d
docker compose ps
docker exec -it cass1 nodetool status
```

4.2. Procedimiento

4.2.1. Experimento 1: caída de nodos

El concepto de la Unidad 1 que se ilustra es la tolerancia a fallos, entendida como la capacidad de seguir atendiendo operaciones cuando una réplica deja de responder, junto con la replicación y el consenso que la sostienen. La pregunta que guio el ensayo fue qué ocurre con las lecturas y las escrituras en cada motor cuando se detiene uno de los tres nodos y cuando solo queda una réplica disponible.

Las hipótesis se formularon antes de los ensayos. En CockroachDB se esperaba que el clúster siguiera confirmando operaciones mientras el rango conservara su quórum, con una posible espera por elección de líder, y que dejara de confirmarlas al detener un segundo nodo. En Apache Cassandra se esperaba que `ONE` y `QUORUM` funcionaran con un nodo caído, que `ALL` fallara por exigir tres respuestas y que, con dos nodos caídos, solo `ONE` pudiera completarse desde la única réplica disponible; esta predicción se apoya en los niveles de consistencia descritos por la documentación [2].

En CockroachDB se verificó primero el estado de los nodos, se creó la base `prueba` con la tabla `datos` y tres filas iniciales, y se consultó el rango que almacena la tabla. Después se detuvo `roach2` y se repitieron lectura y escritura; luego se detuvo `roach3` y se intentó consultar con dos nodos caídos. Por último se iniciaron ambos nodos y se registró la recuperación, incluido un estado anómalo de `roach1` que se corrigió al reiniciarlo.

En Apache Cassandra se levantó el clúster, se comprobó la versión con `nodetool version` y se preparó el mismo esquema con factor de replicación 3. Cada etapa combinó `CONSISTENCY` con lecturas y escrituras separadas, y se verificó qué nodos eran réplicas de la clave consultada con `nodetool getendpoints`. Se detuvo `cass2`, se repitieron las pruebas con los tres niveles, se detuvo también `cass3` y se cerró con el arranque de ambos nodos y una lectura con `ALL`.

Listing 2: Comandos del experimento 1 en CockroachDB (extracto).

```
# Linea base y datos iniciales
docker compose ps
docker exec -it roach1 cockroach node status --certs-dir=/certs --host=roach1:26257
docker exec -it roach1 cockroach sql --certs-dir=/certs --host=roach1:26257

CREATE DATABASE prueba;
USE prueba;
CREATE TABLE datos (id INT PRIMARY KEY, valor STRING);
INSERT INTO datos VALUES (1, 'a'), (2, 'b'), (3, 'c');
SELECT * FROM datos ORDER BY id;
SHOW RANGES FROM TABLE datos;

# Un nodo detenido
docker stop roach2
docker compose ps -a
SELECT * FROM datos ORDER BY id;
INSERT INTO datos VALUES (4, 'un_nodo_caido');
SELECT * FROM datos WHERE id = 4;

# Dos nodos detenidos
docker stop roach3
docker compose ps -a
```

```
SELECT * FROM datos ORDER BY id;

# Recuperacion
docker start roach2 roach3
docker exec -it roach1 cockroach node status --certs-dir=/certs --host=roach1:26257
docker restart roach1
```

Listing 3: Comandos del experimento 1 en Apache Cassandra (extracto).

```
CREATE KEYSPACE prueba WITH replication =
  {'class': 'NetworkTopologyStrategy', 'dc1': 3};
CREATE TABLE prueba.datos (id int PRIMARY KEY, valor text);
CONSISTENCY QUORUM;
INSERT INTO prueba.datos (id, valor) VALUES (1, 'a');
SELECT * FROM prueba.datos;

docker exec cass1 nodetool getendpoints prueba datos 1

# Un nodo detenido
docker stop cass2
docker exec -it cass1 nodetool status

CONSISTENCY ONE;     SELECT * FROM prueba.datos WHERE id = 1;
CONSISTENCY QUORUM;  SELECT * FROM prueba.datos WHERE id = 1;
CONSISTENCY ALL;     SELECT * FROM prueba.datos WHERE id = 1;

# Dos nodos detenidos
docker stop cass3
CONSISTENCY ONE;     INSERT INTO prueba.datos (id, valor) VALUES (7, 'dos_caidos_one')
    ;
CONSISTENCY QUORUM;  INSERT INTO prueba.datos (id, valor) VALUES (8, '
    dos_caidos_quorum');
CONSISTENCY ALL;     INSERT INTO prueba.datos (id, valor) VALUES (9, 'dos_caidos_all')
    ;

# Recuperacion
docker start cass2 cass3
CONSISTENCY ALL;     SELECT * FROM prueba.datos;
```

4.2.2. Experimento 2: partición de red

Este experimento ilustra el teorema CAP: ante una partición, un sistema debe elegir entre mantener la consistencia de lo que confirma o seguir respondiendo en ambos lados. La pregunta fue cómo afecta una partición 2+1 a las lecturas y escrituras de cada motor y qué datos se pueden verificar después de reconectar el nodo aislado.

La hipótesis para CockroachDB fue que el lado mayoritario confirmaría una escritura mientras el rango conservara quórum, y que desde el lado aislado no se confirmaría una escritura nueva. Para Apache Cassandra se esperaba que `ONE` operara en ambos lados si cada uno alcanzaba una réplica, que `QUORUM` solo completara en el lado de dos réplicas y que `ALL` no completara en ninguno [2].

La partición se construyó sin detener los procesos. En cada sistema se desconectó un nodo de la red del proyecto con `docker network disconnect`, se comprobaron las operaciones desde ambos lados y se reconectó el nodo con su dirección IP original. En CockroachDB se aisló `roach3` y se usó una sesión SQL local en ese mismo nodo para el lado aislado. En Apache Cassandra se aisló `cass3` y se abrió `cqlsh` contra `127.0.0.1` para consultar desde el lado aislado.

Listing 4: Aislamiento y reconexión del nodo en cada sistema (extracto).

```
# CockroachDB: aislar roach3 conservando el contenedor encendido
docker network disconnect crdb-lab_crdbnet roach3
docker compose ps -a
docker inspect roach3 --format '{{json .NetworkSettings.Networks}}'
# Lado mayoritario (roach1 y roach2)
SELECT * FROM datos WHERE id = 1;
INSERT INTO datos VALUES (31, 'particion_mayoria');
SELECT * FROM datos WHERE id = 31;
# Lado aislado (dentro de roach3)
docker exec -it roach3 cockroach sql --certs-dir=/certs \
  --host=localhost:26257 --database=prueba
# Reconexion con la IP original
docker network connect --ip 172.22.0.2 --alias roach3 crdb-lab_crdbnet roach3

# Cassandra: aislar cass3 y consultar desde el propio nodo
docker network disconnect cassandra-lab_cassnet cass3
docker exec -it cass3 cqlsh 127.0.0.1 9042
CONSISTENCY ONE;     INSERT INTO prueba.datos (id, valor) VALUES (21, '
    particion_aislado_one');
CONSISTENCY QUORUM;  INSERT INTO prueba.datos (id, valor) VALUES (22, '
    particion_aislado_quorum');

# Reconexion con la IP original
docker network connect --ip 172.28.0.13 cassandra-lab_cassnet cass3
```

4.3. Resultados

4.3.1. Experimento 1 con CockroachDB

La línea base mostró los tres nodos con `is_available=true` e `is_live=true`. La tabla `datos` quedó con las filas 1, 2 y 3, y `SHOW RANGES` indicó el rango 87 con réplicas y réplicas votantes `{1,2,3}` (Figuras 4 y 5). Con `roach2` detenido, la lectura devolvió las tres filas y la escritura de la fila 4 respondió `INSERT 0 1`; la verificación posterior mostró la fila 4, de modo que la operación se confirmó con dos de las tres réplicas votantes (Figura 6). Al detener también `roach3`, la consola SQL de `roach1` no obtuvo resultado y la salida quedó cortada por el ancho de la ventana con el mensaje `replica unavailable`, asociado en la sesión a la pérdida de quórum del rango (Figura 7).

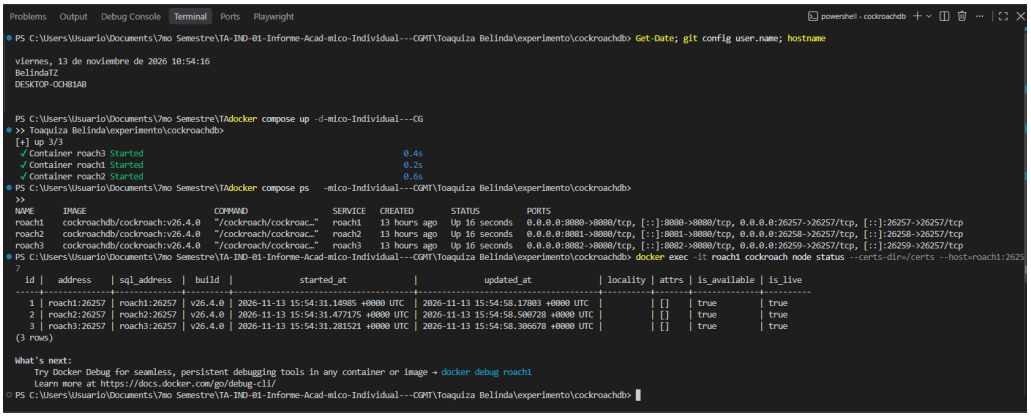


Figura 4: Experimento 1, CockroachDB: línea base con los tres nodos activos y disponibles según `node status`.

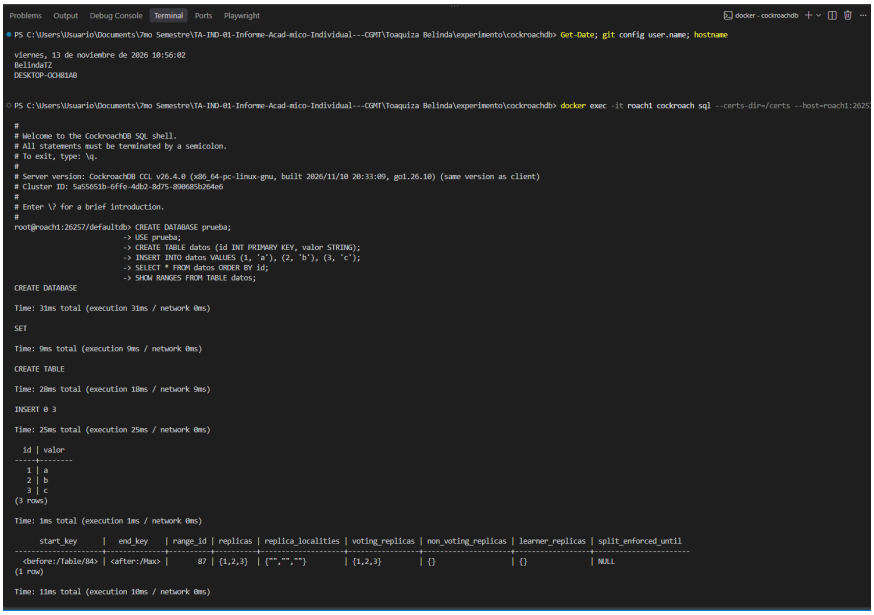


Figura 5: Experimento 1, CockroachDB: creación de la tabla, filas 1 a 3 y rango 87 con réplicas votantes `{1,2,3}`.



Figura 6: Experimento 1, CockroachDB: con `roach2` detenido, la lectura devuelve las filas 1 a 3 y la escritura de la fila 4 se confirma y se verifica.

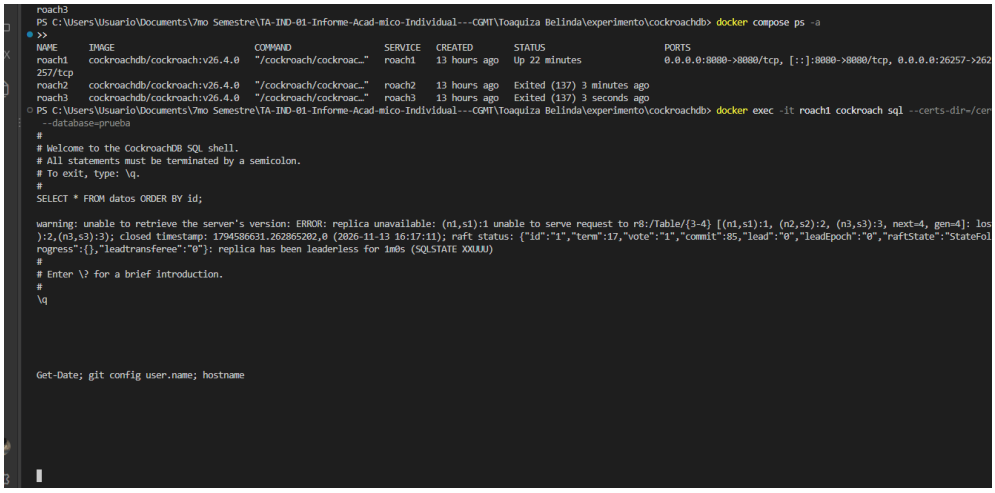


Figura 7: Experimento 1, CockroachDB: con `roach2` y `roach3` detenidos, la consulta devuelve `replica unavailable` por pérdida de quórum.

Al iniciar `roach2` y `roach3`, la consola SQL volvió a responder antes de que `node status` reconociera al nodo 1: las filas 1 a 4 seguían visibles y la fila 6 se insertó y verificó. En ese momento `node status` mostraba `address=NULL`, `is_available=true` e `is_live=false` para `roach1`. Tras `docker restart roach1`, el nodo recuperó su dirección, los tres nodos quedaron activos y la consulta final mostró las filas 1, 2, 3, 4 y 6 sobre el rango 88 con votantes `{1,2,3}`. La causa del estado anómalo y del cambio de identificador del rango no quedó verificada. Estas capturas están en `experimento/comandos.md` (pasos 5 y 6 del experimento).

4.3.2. Experimento 1 con Apache Cassandra

Con los tres nodos `UN` y `QUORUM`, la lectura devolvió las filas iniciales, y `nodetool getendpoints` mostró que la clave 1 residía en las tres direcciones del anillo (Figura 8). Al detener `cass2`, `nodetool status` lo marcó `DN`. Con `ONE` la lectura de la clave 1 respondió y la escritura de la fila 4 se confirmó en una lectura posterior; con `QUORUM` ocurrió lo mismo con la fila 5; con `ALL`, tanto la lectura como la escritura de la fila 6 devolvieron `Unavailable` con `required_replicas: 3` y `alive_replicas: 2` (Figura 9).

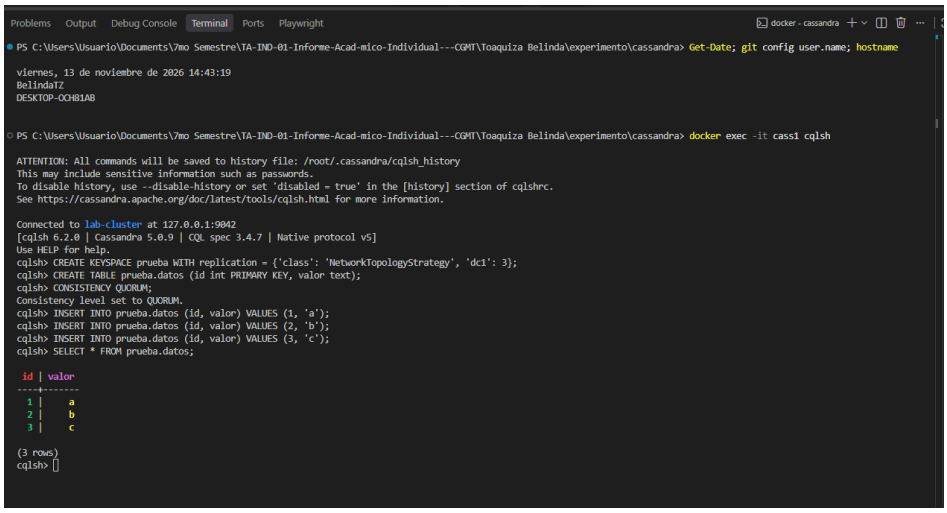


Figura 8: Experimento 1, Apache Cassandra: espacio de claves `prueba` con factor de replicación 3, filas iniciales y lectura con `QUORUM`.



Figura 9: Experimento 1, Apache Cassandra: `cass2` detenido y en estado `DN`; las operaciones con `ONE` y `QUORUM` se completan y `ALL` falla por falta de la tercera réplica.

Detener también `cass3` dejó a `cass1` como único nodo `UN`. En ese estado, `ALL` y `QUORUM` rechazaron lecturas y escrituras con `Unavailable` (tres y dos réplicas requeridas frente a una disponible), mientras `ONE` completó la lectura de la clave 1 y la escritura de la fila 7, que se verificó en la lectura siguiente (Figura 10). Tras `docker start cass2 cass3`, el primer `nodetool status` todavía mostró dos nodos `DN`; sin otro reinicio visible, el estado convergió a tres `UN` y `CONSISTENCY ALL` devolvió seis filas (1, 2, 3, 4, 5 y 7). No aparecieron las filas 6, 8 ni 9, cuyos intentos de escritura ya mostraban errores en las capturas de las etapas previas.

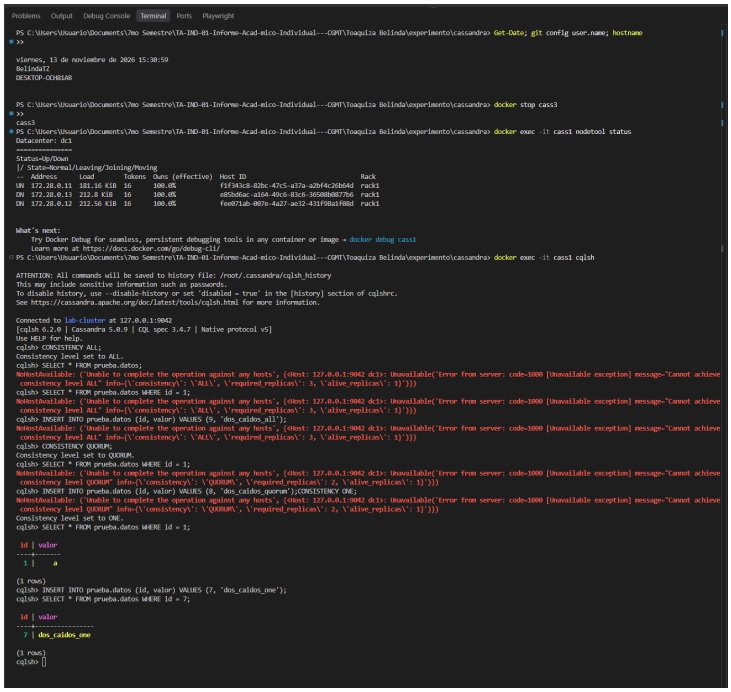


Figura 10: Experimento 1, Apache Cassandra: con dos nodos detenidos, `ALL` y `QUORUM` devuelven `Unavailable` y `ONE` completa la lectura y la escritura de la fila 7.

4.3.3. Experimento 2 con CockroachDB

La línea base mostró los tres nodos activos, las filas 1, 2, 3, 4 y 6 y el rango 88 con votantes `{1,2,3}`; `docker inspect` confirmó que `roach3` usaba la red del proyecto con la dirección `172.22.0.2`. Al desconectarlo, la inspección devolvió `{}` y `docker compose ps -a` siguió mostrando el contenedor en ejecución: el proceso estaba encendido y la conectividad aislada (Figura 11). Del lado mayoritario, la lectura de la clave 1, la escritura de la fila 31 y su verificación se completaron con dos nodos conectados (Figura 12).

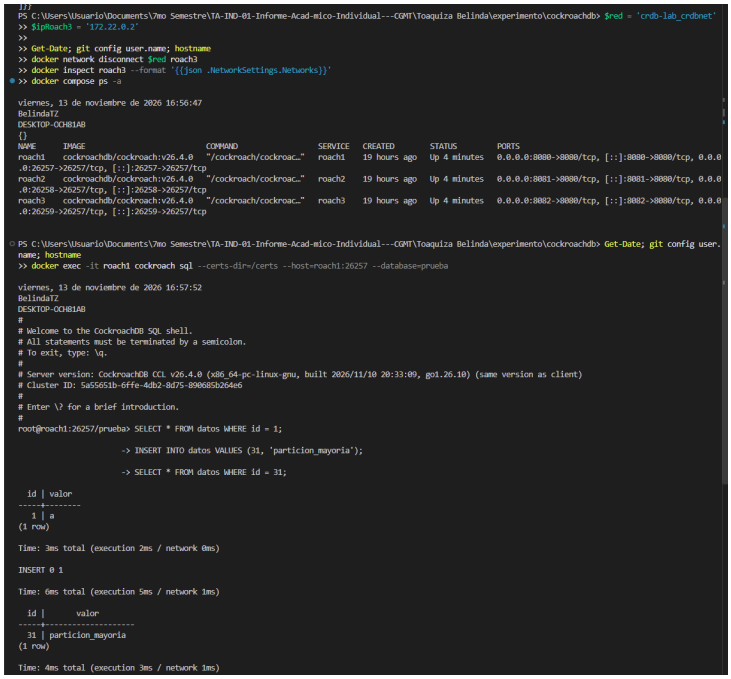


Figura 11: Experimento 2, CockroachDB: `roach3` aislado de la red pero con el contenedor en ejecución, y operaciones confirmadas desde el lado mayoritario.

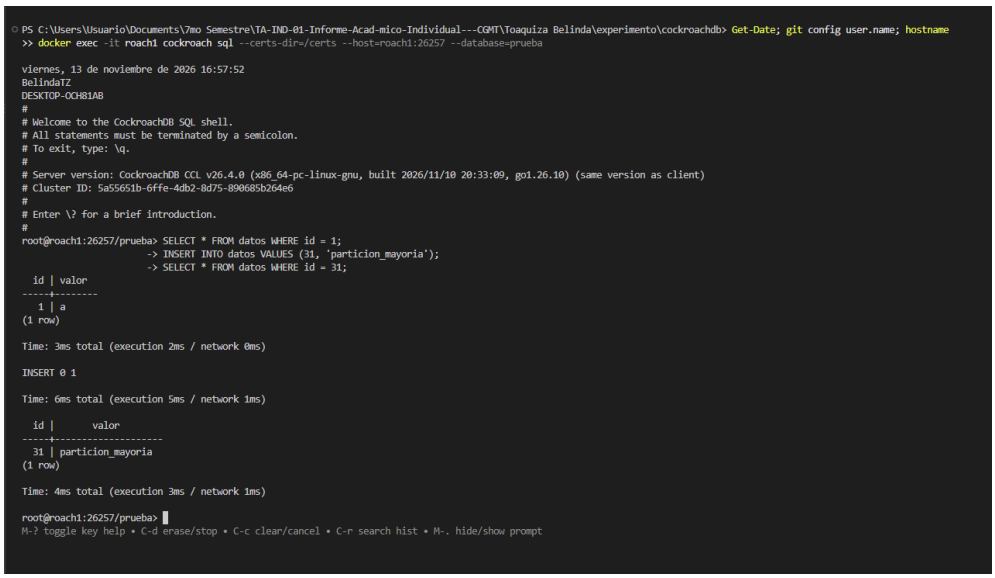


Figura 12: Experimento 2, CockroachDB: lectura de la clave 1, `INSERT 0 1` de la fila 31 y verificación de esa fila desde `roach1`.

Desde la sesión SQL local de `roach3` aislado, la lectura de la clave 1 devolvió `replica unavailable` con el detalle `lost quorum` y `SQLSTATE XXUUU`; a continuación se introdujo la escritura de la fila 32, pero la captura termina sin respuesta ni nuevo indicador de consulta, por lo que el resultado de esa escritura no quedó confirmado (Figura 13). El primer intento de reconexión falló porque los argumentos de `docker network connect` se escribieron en orden incorrecto; el segundo usó la dirección original y a las 17:03 los tres nodos aparecieron activos, la fila 31 seguía visible y el rango conservaba los votantes `{1,2,3}` (Figura 14). En una comprobación posterior, dos consultas de la fila 32 no devolvieron filas, lo que no permite concluir qué ocurrió con el intento aislado.

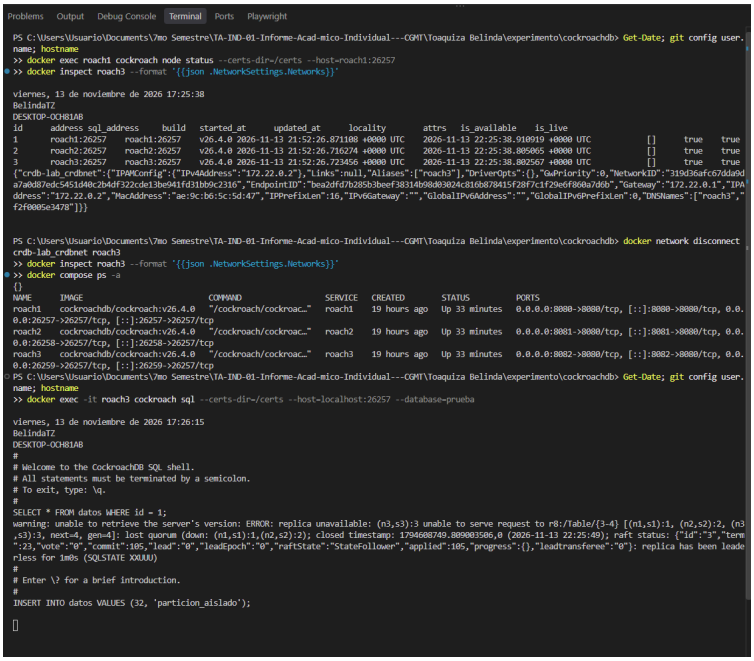


Figura 13: Experimento 2, CockroachDB: sesión SQL en `roach3` aislado; la lectura de la clave 1 falla con `lost quorum` y la escritura de la fila 32 queda sin respuesta visible.

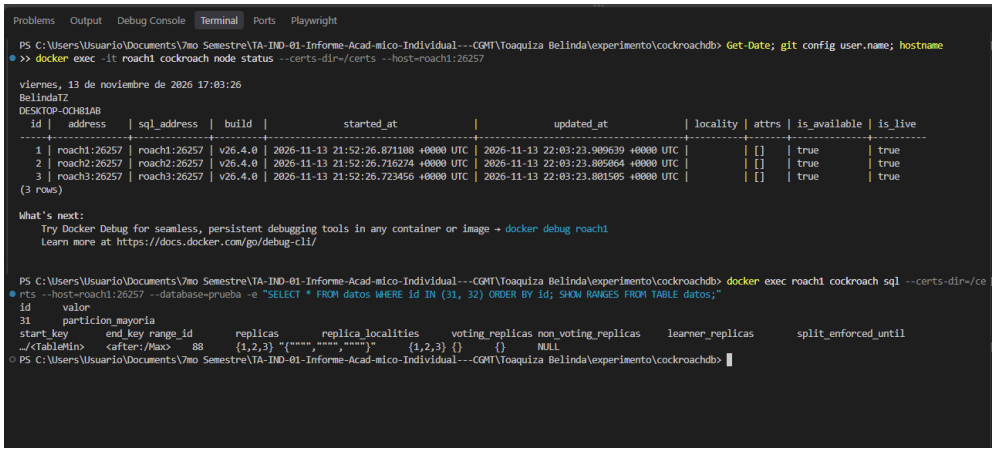


Figura 14: Experimento 2, CockroachDB: reconexión de `roach3` con su IP original, tres nodos activos y fila 31 visible.

4.3.4. Experimento 2 con Apache Cassandra

A las 16:25:51 `docker compose ps` todavía mostraba `cass3` como `Up (healthy)`, mientras `nodetool status` ya lo reportaba `DN`; la inspección de redes no listó ninguna y un segundo `disconnect` confirmó que el contenedor ya estaba aislado (Figura 15). Desde `cqlsh` contra `127.0.0.1`, `ONE` completó la lectura de la clave 1 y la escritura de la fila 21, que se verificó de inmediato; `QUORUM` rechazó lectura y escritura con `required_replicas: 2` y `alive_replicas: 1` (Figura 16). Del lado de `cass1` y `cass2`, `ONE` y `QUORUM` leyeron la clave 1 y registraron las filas 23 y 24, mientras `ALL` falló con `required_replicas: 3` y `alive_replicas: 2` (Figura 17).

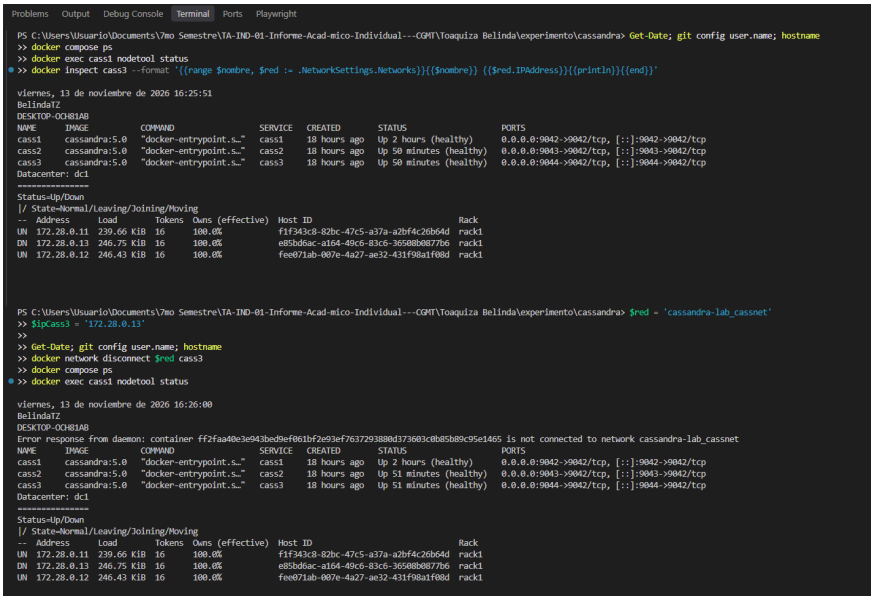


Figura 15: Experimento 2, Apache Cassandra: `cass3` aislado de la red y confirmación del `disconnect` mientras el proceso seguía activo.

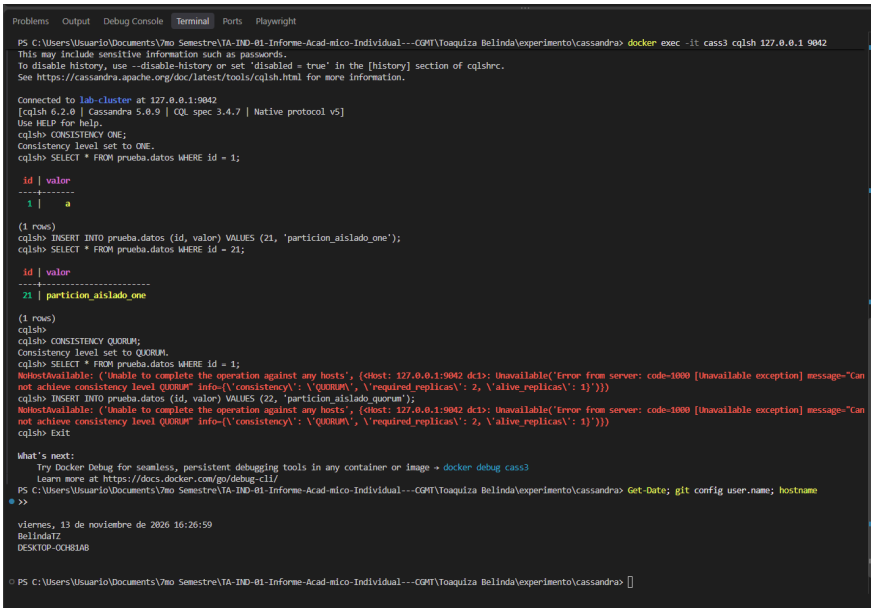


Figura 16: Experimento 2, Apache Cassandra: lado aislado; `ONE` completa la escritura de la fila 21 y `QUORUM` falla por falta de la segunda réplica.

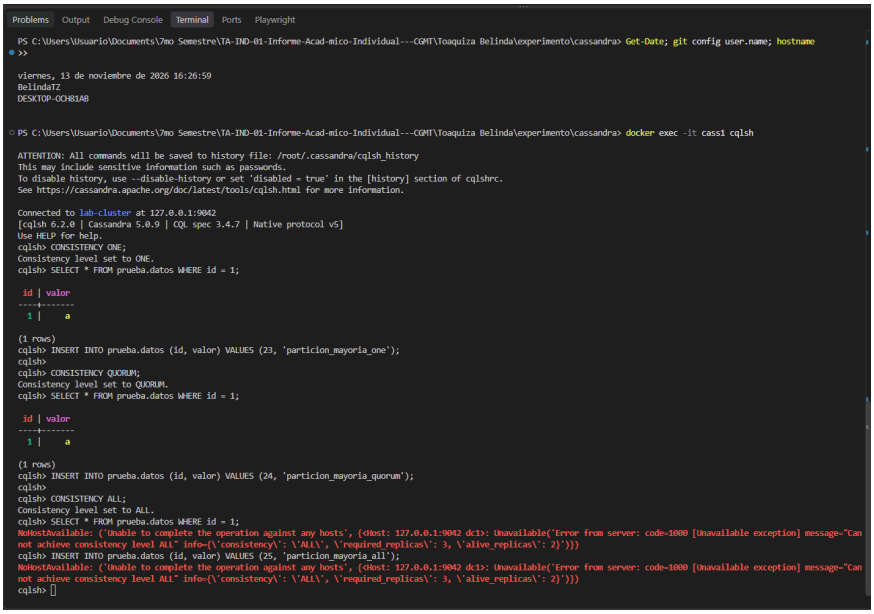


Figura 17: Experimento 2, Apache Cassandra: lado mayoritario; `ONE` y `QUORUM` operan y `ALL` falla por requerir tres réplicas.

La reconexión se hizo con la dirección `172.28.0.13` y, tras ella, `nodetool status` mostró los tres nodos `UN`. Con `CONSISTENCY ALL`, la consulta de las claves 21 a 26 devolvió exactamente las filas 21, 23 y 24 (Figura 18). Las filas 22 y 25 habían devuelto error durante la partición y la 26 se consultó sin haberse intentado escribir. Como se usaron claves distintas, la evidencia muestra la disponibilidad por nivel y la visibilidad posterior, pero no prueba la reconciliación de escrituras concurrentes sobre una misma clave.

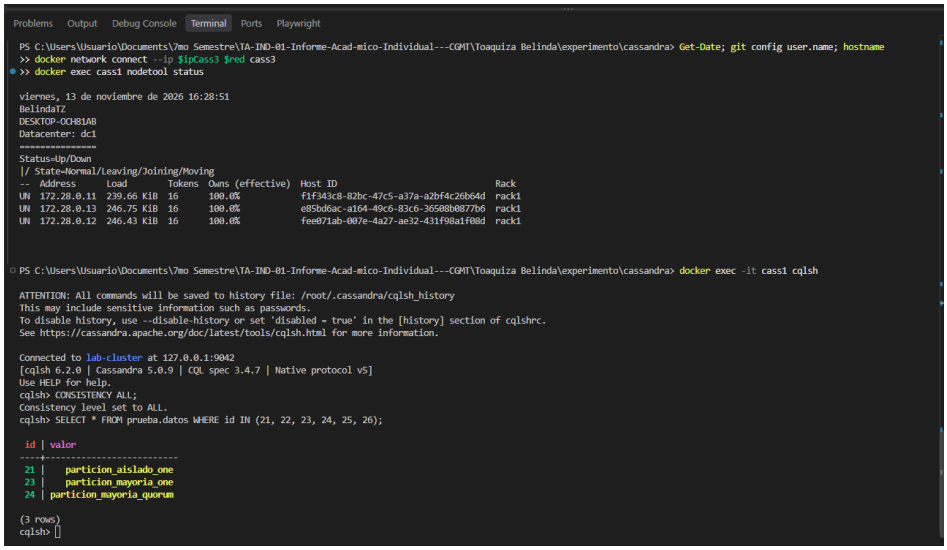


Figura 18: Experimento 2, Apache Cassandra: reconexión de `cass3`, tres nodos `UN` y lectura con `ALL` que devuelve las filas 21, 23 y 24.

La Tabla 1 resume el primer experimento y la Tabla 2 el segundo, con los resultados observados en cada sistema.

Tabla 1: Resultados del experimento 1 (caída de nodos).

Situación	CockroachDB	Apache Cassandra
Todos los nodos activos	Tres nodos disponibles; rango 87 con votantes `{1,2,3}`.	Tres nodos `UN` en `dc1`, RF=3; filas 1 a 3.
Un nodo caído	Lectura y escritura continúan; la fila 4 persiste.	`ONE` y `QUORUM` operan; `ALL` falla (3 requeridas, 2 vivas).
Dos nodos caídos	`replica unavailable` por pérdida de quórum; la salida aparece parcialmente en la captura.	`ONE` opera; `QUORUM` y `ALL` fallan con 1 réplica viva.
Tras recuperar	Reinicio de `roach1`; filas 1, 2, 3, 4 y 6 y tres nodos activos.	Tres nodos `UN`; `ALL` devuelve seis filas (1, 2, 3, 4, 5 y 7).

Tabla 2: Resultados del experimento 2 (partición de red 2+1).

Situación	CockroachDB	Apache Cassandra
Lado aislado	Lectura con `lost quorum`; la escritura de la fila 32 queda sin respuesta visible.	`ONE` opera; `QUORUM` falla (2 requeridas, 1 viva); `ALL` no se probó.
Lado mayoritario	Lectura de la clave 1 y fila 31 confirmada y visible.	`ONE` y `QUORUM` operan; `ALL` falla (3 requeridas, 2 vivas).
Tras reconectar	Tres nodos activos, fila 31 visible y votantes `{1,2,3}`.	Tres nodos `UN`; `ALL` devuelve 21, 23 y 24.

4.4. Interpretación y conclusiones del experimento

- **Contraste con las hipótesis.** Los resultados coincidieron con lo previsto en los comportamientos observados. CockroachDB mantuvo lectura y escritura cuando el rango conservó la mayoría y dejó de confirmar la consulta al perderla. En Apache Cassandra, con un nodo caído, `ONE` y `QUORUM` se completaron y `ALL` falló; con dos nodos caídos solo `ONE` operó. En la partición, `ONE` funcionó en ambos lados, `QUORUM` solo en el mayoritario y `ALL` falló allí donde se probó.

- **Diferencia de mecanismo.** CockroachDB aplica una regla única de quórum por rango, de modo que la disponibilidad depende de cuántas réplicas votantes conserva la mayoría [1]. Apache Cassandra traslada esa decisión al nivel de consistencia de cada operación, que el cliente puede cambiar entre consultas [2], [6]. Por eso, con la misma topología de tres réplicas, la disponibilidad observada ante dos nodos caídos fue distinta: el primero dejó de confirmar y el segundo pudo responder con `ONE`.

- **Qué muestran las particiones.** El aislamiento con `docker network disconnect` reprodujo el escenario de CAP sin detener los procesos: el nodo aislado siguió en ejecución y el lado mayoritario conservó la capacidad de confirmar operaciones. En CockroachDB el lado aislado no pudo leer por `lost quorum`; en Apache Cassandra pudo leer y escribir con `ONE`, lo que evidencia el sacrificio de consistencia que ese nivel acepta. La lectura posterior con `ALL` devolvió los datos escritos en ambos lados, pero el ensayo no identifica el mecanismo de convergencia ni el tiempo exacto de detección, que en Apache Cassandra depende de gossip y de mecanismos de reparación [11], [16].

- **Conclusiones del experimento.** La evidencia respalda el contraste descrito en la sección 3: CockroachDB se comportó como sistema CP en las operaciones probadas, porque ante la partición conservó la consistencia de lo que confirmaba y rechazó la lectura en la minoría; Apache Cassandra se comportó como AP con `ONE` y como CP con `QUORUM` y `ALL`, según el nivel elegido por la aplicación. El experimento mide disponibilidad y consistencia observada, no rendimiento: no se registraron latencias ni se probaron escrituras concurrentes sobre una misma clave.

4.5. Limitaciones y reproducibilidad

Los tres nodos de cada motor se ejecutaron como contenedores en una sola máquina Windows. La prueba ejercita detenciones de procesos y aislamiento de red entre contenedores, pero no representa fallas de equipos físicos ni las condiciones de una red distribuida entre máquinas. Se probaron una topología, unas

versiones y unas operaciones concretas, y los resultados no se generalizan a otras configuraciones. No se midieron tiempos de respuesta ni la duración exacta de detección y recuperación.

Para reproducir el ensayo se requieren Docker en Windows, PowerShell y los archivos de composición de `experimento/`. En CockroachDB hay que generar los certificados en el volumen `crdb-certs` e inicializar el clúster una sola vez; en Apache Cassandra hay que esperar hasta que `nodetool status` muestre las tres direcciones como `UN`. Los pasos, las horas, las direcciones IP y las salidas completas de cada etapa están en `experimento/comandos.md`, que también registra las capturas de evidencia que no se incluyen en este informe.

5. Ejemplos del entorno del estudiante

Los dos casos siguientes se observaron en servicios de uso cotidiano y muestran, en la interfaz, el mismo compromiso que se estudió en los motores distribuidos: una operación se completa, pero su efecto no aparece de inmediato en todas las vistas de la aplicación. No se conoce la arquitectura interna de esos servicios, por lo que la explicación se plantea como una hipótesis compatible con lo observado y con los conceptos de la Unidad 1.

5.1. Ejemplo 1: aplicación móvil de Banco Pichincha

En una ocasión recibí una transferencia y la persona que la realizó me envió el comprobante. Al abrir el historial de transacciones de la aplicación, el pago no aparecía. Sin embargo, al buscar por el número de transferencia del comprobante, el sistema mostró el registro como válido. Es decir, la operación existía y estaba registrada, pero una de las vistas de la aplicación no la reflejaba en ese momento.

El caso corresponde a una lectura que no observa una escritura ya confirmada, es decir, a una lectura desactualizada. La transparencia de distribución explica por qué la usuaria no puede advertirlo: la aplicación no informa qué réplica o servicio atendió cada consulta, y el historial y la búsqueda por número pueden resolverse por rutas distintas, una contra una copia todavía no actualizada y otra contra el registro autoritativo [6], [7]. Bajo el teorema CAP, el sistema prefirió responder con la copia disponible antes que bloquear la consulta o devolver un error, y asumió el riesgo de mostrar un estado anterior. En los términos del experimento, es el mismo compromiso que se observó en E2: una lectura con garantía fuerte se habría detenido ante la duda, mientras que una lectura con menor garantía podía devolver la vista previa [1], [6].

La lección del caso es que la consistencia no se percibe en abstracto, sino en la diferencia entre la durabilidad de un dato y su visibilidad. La transferencia no se perdió: seguía disponible por su identificador. Lo que falló fue la coherencia entre dos vistas del mismo sistema, y esa incoherencia se interpreta como un error de la aplicación aunque el dato esté a salvo. Un diseño que exige quórum para leer, como el de CockroachDB en las operaciones probadas, habría devuelto un error antes que una lista incompleta; uno que permite `ONE`, como Apache Cassandra, puede devolver la copia desactualizada [1], [2]. Para la persona usuaria, la diferencia entre ambos comportamientos no es técnica: en un caso ve «no existe» y en el otro ve «error», y ambas respuestas exigen una decisión distinta.

5.2. Ejemplo 2: plataforma de capacitaciones del CNE

En la plataforma de capacitaciones en línea del CNE realicé una prueba sencilla. Al terminarla, el sistema no la marcó como completada. La repetí tres veces ese mismo día y el estado no cambió; al día siguiente volví a responderla y entonces sí quedó registrada. La prueba era la misma y el resultado no varió: lo que cambió fue el momento en que se consultó el estado.

El avance de una capacitación es una escritura de estado que debe quedar visible para la persona que la completó. Si la pantalla de progreso consulta una réplica o un servicio distinto del que recibe el intento,

y la confirmación se propaga de forma asíncrona, la interfaz puede mostrar un estado anterior aunque el evento ya exista; ese comportamiento corresponde a la consistencia eventual [6], [7]. La repetición manual actuó como un reintento del cliente y, además, permitió comprobar un requisito de diseño: la operación debería ser idempotente para que repetirla no duplique el registro ni altere la calificación. La convergencia observada al día siguiente es análoga a la que los motores distribuidos logran mediante mecanismos de reparación que comparan réplicas e intercambian diferencias, aunque en este caso no es posible saber si el cambio provino de una reparación, de una caché renovada o de un proceso interno del servicio [11].

El caso se relaciona con la tolerancia a fallos de la sección 4: en E1, tras volver a iniciar los nodos, el primer `nodetool status` todavía mostró dos nodos `DN` y el estado convergió después; la persona usuaria de una aplicación ve algo equivalente cuando el progreso tarda en actualizarse. La diferencia es que en el clúster la demora se explica por la propagación del estado entre réplicas, y en la plataforma solo se observa el efecto. Por eso, un sistema que privilegia la disponibilidad y la latencia debe comunicar el estado «pendiente de sincronización» en lugar de mostrar «no completado», y debe aceptar que un reintento sea seguro.

Ambos ejemplos confirman que la transparencia de distribución tiene un límite práctico: el sistema puede ocultar dónde están los datos, pero no puede ocultar que dos vistas del mismo hecho no coinciden. La coherencia entre vistas, más que la ubicación de las réplicas, es lo que la persona usuaria percibe como confiabilidad [1], [2].

6. Análisis comparativo

Tabla 3: Comparación entre CockroachDB y Apache Cassandra.

Criterio	CockroachDB	Apache Cassandra
Transparencias de distribución	Acceso SQL y ubicación lógica de rangos quedan mayormente ocultos; la pérdida de quórum y los reintentos sí llegan a la aplicación [1], [3], [9].	CQL y tokens ocultan la ubicación, pero el cliente configura el nivel de consistencia y puede recibir errores de disponibilidad [2], [10].
CAP y PACELC	En E2, la mayoría confirmó una escritura; la réplica aislada no pudo completar la lectura actual. Perfil CP y PC/EC para las operaciones actuales, como inferencia del quórum Raft [1].	E2: ONE operó en ambos lados, QUORUM solo con dos réplicas y ALL falló en la mayoría. La elección del nivel cambia disponibilidad y coordinación; no se midió latencia [2].
Modelo de comunicación	Cliente-servidor con SQL/PostgreSQL; los nodos coordinan rangos mediante Raft y esperan confirmación de quórum para escrituras [1], [9].	Cliente-servidor con CQL hacia un coordinador y comunicación entre pares para réplicas; la respuesta espera el nivel elegido y gossip opera en segundo plano [2], [10], [17].
Replicación y consenso	Replicación por rangos con consenso Raft; se requieren dos de tres votantes para conservar el quórum del rango [1].	Replicación por estrategia de clave de espacio y factor RF=3; los niveles ONE, QUORUM y ALL determinan respuestas requeridas, sin consenso Raft global [2].
Tolerancia a fallos	E1: con un nodo detenido hubo lecturas y escrituras; con dos, se perdió quórum. E2: la mayoría escribió y la lectura actual aislada falló [1].	E1 y E2 mostraron disponibilidad dependiente del nivel y de las réplicas alcanzables. Tras reconectar, se verificaron tres nodos UN; el mecanismo exacto de convergencia no se midió [2], [11].

La diferencia práctica principal es dónde se fija el compromiso de disponibilidad. CockroachDB conserva una regla de quórum Raft para las operaciones actuales; Cassandra permite seleccionar el número de réplicas que deben responder por operación. Los ensayos concuerdan con esa diferencia en las condiciones probadas: CockroachDB dejó de confirmar operaciones al perder mayoría, mientras Cassandra permitió operaciones `ONE` en ambos lados de la partición. Los resultados no miden rendimiento ni prueban escrituras concurrentes sobre la misma clave [1], [2].

7. Postura crítica

7.1. Fortalezas

La fortaleza central de CockroachDB es que conserva una interfaz SQL y un modelo relacional sobre un almacenamiento distribuido, de modo que la aplicación no elige un nivel de consistencia por operación ni necesita conocer la ubicación de los datos: la capa de rangos y el consenso Raft resuelven ambas decisiones [1], [3]. Esa separación reduce el riesgo de configurar mal una lectura o una escritura, porque la coherencia es la regla predeterminada y no una opción que el cliente pueda omitir. En el ensayo, el clúster confirmó lecturas y escrituras con un nodo detenido y mantuvo la fila 31 visible después de reconectar el nodo aislado, sin que la sesión SQL tuviera que cambiar de configuración entre etapas.

Sobre esa base, CockroachDB incorpora mecanismos pensados para la operación geográfica, como las tablas multi-región y los arrendamientos de líder [4], [15], además de las lecturas de seguidor que permiten atender consultas en una réplica sin esperar al líder [12]. El experimento no ejercitó esas funciones, pero muestran que el mismo modelo de quórum puede extenderse a despliegues distribuidos sin cambiar el contrato de la aplicación. La contraparte es que la coherencia se apoya en una mayoría por rango y no en una réplica privilegiada, lo que evita depender de un nodo concreto y explica por qué el clúster siguió respondiendo cuando se detuvo `roach2`.

La fortaleza simétrica de Apache Cassandra es la disponibilidad graduable. El nivel de consistencia de cada operación fija cuántas réplicas deben responder y, por tanto, cuánta tolerancia se acepta frente a nodos caídos o aislados [2], [6]. En E1, con dos nodos detenidos, `ONE` completó la lectura y la escritura de la fila 7; en E2, el lado aislado pudo escribir la fila 21 y el lado mayoritario las filas 23 y 24. Esa capacidad de seguir operando con una sola réplica disponible es la ventaja que el modelo de CockroachDB no ofrece en las operaciones probadas. A ello se suman el particionamiento por tokens, que permite crecer agregando nodos, y los mecanismos de reparación entre réplicas [11], [16]. El caso de Discord, con un clúster de 177 nodos y trillones de mensajes, confirma que el modelo sostiene operación a gran escala, aunque también documenta el costo de mantenerlo [5].

Ambas fortalezas son condicionales y no comparables en abstracto: la de CockroachDB es la coherencia sin configuración, y la de Apache Cassandra, la disponibilidad que el cliente decide operación por operación. El contraste de la sección 6 y los resultados de la sección 4 respaldan esa lectura, porque la diferencia observada no provino de la topología, que fue la misma de tres réplicas, sino de dónde se fija el compromiso entre consistencia y disponibilidad [1], [6].

7.2. Limitaciones y compromisos (*trade-offs*)

CockroachDB paga su coherencia con disponibilidad y con coordinación. Cuando el rango perdió la mayoría, la consola SQL dejó de obtener resultado con el mensaje `replica unavailable`, y la sesión aislada de E2 falló con `lost quorum` y `SQLSTATE XXUUU`; en ambos casos el sistema prefirió no confirmar antes que devolver un dato sin respaldo mayoritario [1]. Cada escritura confirmada exige el acuerdo de dos de las tres réplicas votantes, de modo que el costo en latencia crece con la distancia entre nodos, y el modelo PACELC permite anticipar que, sin partición, el sistema tiende a privilegiar la consistencia sobre la latencia [2]. A eso se añade un costo operativo propio: los certificados TLS, la inicialización del clúster y la capa de metadatos que reubica los rangos. En E1, tras la caída de dos nodos, `node status` mostró a `roach1` con `address=NULL` e `is_live=false`, y el rango cambió de identificador 87 a 88; la captura documenta el hecho, pero no permite atribuirlo a una causa concreta.

En Apache Cassandra la contrapartida se traslada al cliente y a la operación. Como el nivel de consistencia se elige por consulta, una aplicación puede leer datos desactualizados con `ONE` o dejar de operar con `ALL` cuando falta una réplica; en E1, `ALL` devolvió `Unavailable` con tres réplicas requeridas y dos vivas, y en E2 lo mismo ocurrió en el lado mayoritario [6]. La convergencia posterior depende de mecanismos de

mejor esfuerzo y de tareas de reparación [11]: en E2, la fila 21 escrita en el lado aislado solo se verificó después de reconectar, la fila 22 no apareció en la lectura final y la 26 se consultó sin haberse escrito. El ensayo no permite saber si medió hints, reparación o ambos. La literatura reporta además que el nivel de consistencia afecta el rendimiento y el consumo energético [7], [13], [14], pero aquí no se registraron latencias, de modo que esa dimensión queda fuera de las conclusiones.

Los dos motores comparten un límite del estudio y un costo que conviene declarar. La prueba se ejecutó en una sola máquina, con contenedores, y midió disponibilidad y consistencia observadas, no rendimiento ni comportamiento ante fallas de hardware. Operar cualquiera de los dos exige personal que entienda su modelo: en CockroachDB, la gestión de certificados y del consenso por rango; en Apache Cassandra, la elección del nivel por operación y la ejecución periódica de `nodetool repair`. Elegir uno sin ese conocimiento traslada el riesgo del motor a la aplicación [3], [16].

7.3. Contextos en los que no sería la mejor opción

CockroachDB no sería la mejor opción cuando la prioridad es que ninguna región deje de aceptar escrituras durante una partición o cuando la latencia de escritura es más importante que la uniformidad inmediata del dato. En esos escenarios, la regla de mayoría por rango convierte la disponibilidad en rehén de la conectividad, tal como ocurrió en E2: el lado aislado no pudo leer ni confirmar la escritura mientras el lado mayoritario seguía operando [1]. Tampoco sería la elección más económica para cargas de escritura masiva con claves simples, donde el costo de coordinar cada rango puede superar el beneficio de las transacciones y del SQL, ni para equipos que no puedan asumir la operación de certificados y de un clúster con consenso.

Apache Cassandra no sería la mejor opción cuando la aplicación necesita transacciones que abarcan varias filas, unicidad, integridad referencial o lecturas lineales por defecto, porque esas garantías no forman parte de su modelo de consistencia tunable [6]. En E1 y E2, las operaciones con `QUORUM` y `ALL` fallaron en cuanto faltó una réplica, de modo que un sistema que exige siempre esa garantía no gana disponibilidad y sí hereda la complejidad de elegir el nivel. Tampoco sería adecuado si el equipo no puede sostener la reparación y el monitoreo del anillo, o si el volumen de datos cabe en un nodo y no justifica la operación distribuida.

La comparación tampoco favorece a ninguno de los dos en todos los contextos. Para volúmenes que caben en un servidor, un motor relacional centralizado evita la complejidad de ambos; para clústeres de bajo consumo, la literatura muestra que la elección depende más del ajuste entre carga y recursos que de la etiqueta del motor [13]. La postura de este informe es, por tanto, condicional: CockroachDB cuando la coherencia y el SQL son requisito, Apache Cassandra cuando la disponibilidad por operación lo es, y ninguno cuando el problema no exige distribución [1], [2].

8. Conclusiones

El análisis documental y los dos experimentos permiten sostener cinco conclusiones concretas.

- **El compromiso se decide en lugares distintos.** En CockroachDB la regla de mayoría por rango fija la disponibilidad del sistema completo para las operaciones actuales [1]; en Apache Cassandra la decide el cliente al elegir `ONE`, `QUORUM` o `ALL` para cada operación [6]. Esa diferencia de diseño explica los resultados mejor que cualquier diferencia de topología, porque ambos clústeres usaron tres réplicas.
- **La tolerancia a fallos observada coincide con el modelo.** CockroachDB sostuvo lectura y escritura con un nodo detenido y dejó de confirmar al perder la mayoría; Apache Cassandra operó con `ONE` incluso con dos nodos detenidos y en ambos lados de la partición, y falló con `QUORUM` y

`ALL` cuando no reunió las respuestas requeridas. La predicción formulada antes de cada ensayo se cumplió en las operaciones probadas.

- **La transparencia tiene un límite práctico.** Ambos motores ocultan la ubicación de los datos, pero no la falla: la pérdida de quórum llega a la aplicación como `replica unavailable` o `Unavailable`. La diferencia es que CockroachDB exige resolverla en el clúster, mientras que Apache Cassandra permite que la aplicación la enfrente con un nivel menor, a cambio de aceptar datos posiblemente desactualizados [2], [3].
- **No hay ganador absoluto, sino ajuste al requisito.** Si el sistema requiere transacciones, SQL y coherencia por defecto, CockroachDB evita trasladar esa decisión al código; si requiere seguir aceptando escrituras durante una partición y puede tolerar convergencia posterior, Apache Cassandra ofrece esa flexibilidad [4], [11]. La elección inversa obliga a construir en la aplicación lo que el otro motor ya resuelve.
- **El alcance de la evidencia es acotado.** Los ensayos se hicieron en una sola máquina, con tres contenedores por motor, y no midieron latencia ni escrituras concurrentes sobre una misma clave. Por eso las conclusiones se limitan a disponibilidad y consistencia observadas; afirmar cuál es más rápido exigiría un diseño experimental con mediciones temporales y cargas controladas [7], [13].

En conjunto, la evidencia respalda la postura de la sección 7: CockroachDB es una opción sólida cuando la coherencia y el modelo relacional son requisitos, y Apache Cassandra cuando la disponibilidad graduable y la escala de escritura lo son. La decisión responsable no depende de la etiqueta del motor, sino de qué se está dispuesto a ceder cuando la red falla.

Declaración de uso de inteligencia artificial

Para la elaboración de este informe utilicé asistentes de inteligencia artificial generativa basados en modelos de lenguaje, en particular un agente de programación (OpenCode, con el modelo DeepSeek V4.1 Flash) y asistentes conversacionales, durante noviembre de 2026. Su uso se limitó a las tareas de apoyo que se detallan a continuación.

- **Búsqueda de fuentes.** La inteligencia artificial propuso documentos técnicos y artículos académicos para sustentar la comparación. Revisé manualmente cada fuente, comprobé que el contenido citado existiera en el documento original y verifiqué que la referencia correspondiera a la publicación indicada antes de incorporarla a `Referencias/referencias.bib`.
- **Creación de plantillas.** Se empleó para generar las plantillas del informe en LaTeX y los archivos de trabajo en Markdown, incluida la estructura de secciones, el formato de las tablas, los listados de comandos y la organización de la bitácora.
- **Comandos y resolución de errores.** Durante la experimentación se usó para proponer comandos de Docker, `nodetool` y la consola SQL, y para interpretar mensajes de error y consideraciones de configuración. Cada comando se ejecutó en mi equipo y su salida se contrastó con las capturas de evidencia; las decisiones sobre qué probar y cómo interpretar los resultados fueron mías.
- **Redacción formal.** A partir del contenido de los experimentos, de mis resultados y de mis propias palabras, la inteligencia artificial colaboró en la redacción formal del informe, en la organización de los argumentos y en la corrección de estilo. La IA no generó datos experimentales ni sustituyó la ejecución de las pruebas.

Los experimentos, la selección final de las fuentes, la interpretación de los resultados y las decisiones del informe son de mi autoría. Verifiqué que cada referencia citada existiera y que las cifras y salidas mencionadas coincidieran con las capturas; cuando una afirmación no tenía respaldo en la evidencia, la retiré o la señalé como no verificada.

Referencias

[1] Cockroach Labs. «Replication Layer,» visitado 13 de nov. de 2026. dirección: https://docs.cockroachlabs.com/docs/stable/architecture/replication-layer

[2] Apache Software Foundation. «Dynamo,» Apache Cassandra Documentation, visitado 13 de nov. de 2026. dirección: https://cassandra.apache.org/doc/stable/cassandra/architecture/dynamo.html

[3] Cockroach Labs. «Architecture Overview,» visitado 13 de nov. de 2026. dirección: https://docs.cockroachlabs.com/docs/stable/architecture/overview

[4] N. VanBenschoten et al., «Enabling the Next Generation of Multi-Region Applications with CockroachDB,» en *Proceedings of the 2022 International Conference on Management of Data*, Association for Computing Machinery, 2022. DOI: 10.1145/3514221.3526053 dirección: https://dl.acm.org/doi/10.1145/3514221.3526053

[5] B. Ingram. «How Discord Stores Trillions of Messages,» Discord Engineering, visitado 13 de nov. de 2026. dirección: https://discord.com/blog/how-discord-stores-trillions-of-messages

[6] Apache Software Foundation. «Guarantees,» Apache Cassandra Documentation, visitado 13 de nov. de 2026. dirección: https://cassandra.apache.org/doc/stable/cassandra/architecture/guarantees.html

[7] S. Ferreira, J. Mendonça, B. Nogueira, W. Tiengo y E. Andrade, «Impacts of data consistency levels in cloud-based NoSQL for data-intensive applications,» *Journal of Cloud Computing*, vol. 13, 2024, **artno** 158. DOI: 10.1186/s13677-024-00716-7 dirección: https://link.springer.com/article/10.1186/s13677-024-00716-7

[8] Cockroach Labs. «Storage Layer,» visitado 13 de nov. de 2026. dirección: https://docs.cockroachlabs.com/docs/stable/architecture/storage-layer

[9] Cockroach Labs. «Developer Basics,» visitado 13 de nov. de 2026. dirección: https://docs.cockroachlabs.com/docs/stable/developer-basics

[10] Apache Software Foundation. «Native Protocol Versions,» Apache Cassandra Documentation, visitado 13 de nov. de 2026. dirección: https://cassandra.apache.org/doc/stable/cassandra/reference/native-protocol.html

[11] Apache Software Foundation. «Repair,» Apache Cassandra Documentation, visitado 13 de nov. de 2026. dirección: https://cassandra.apache.org/doc/stable/cassandra/managing/operating/repair.html

[12] Cockroach Labs. «Follower Reads,» visitado 13 de nov. de 2026. dirección: https://docs.cockroachlabs.com/docs/stable/follower-reads

[13] L. F. da Silva y J. V. F. Lima, «An evaluation of relational and NoSQL distributed databases on a low-power cluster,» *The Journal of Supercomputing*, vol. 79, n.º 12, págs. 13 402-13 420, 2023. DOI: 10.1007/s11227-023-05166-7 dirección: https://link.springer.com/article/10.1007/s11227-023-05166-7

[14] H. Nejati SharifAldin y F. Nayebi Pour, «Middleware-enforced Timed Causal Consistency for Apache Cassandra: An energy–performance–consistency evaluation against static consistency levels using YCSB,» *Software Impacts*, vol. 28, 2026, **artno** 100817. DOI: 10.1016/j.simpa.2026.100817 dirección: https://www.sciencedirect.com/science/article/pii/S2665963826000072

[15] I. Kettaneh, T. Radeva, A. Ajmani, S. Bhola, N. VanBenschoten y R. Taft, «Scalable Leader Leases For Multi Consensus Groups in CockroachDB,» en *Companion of the International Conference on Management of Data*, Association for Computing Machinery, 2026. DOI: 10.1145/3788853.3803081 dirección: https://dl.acm.org/doi/10.1145/3788853.3803081

[16] Apache Software Foundation. «Architecture,» Apache Cassandra Documentation, visitado 13 de nov. de 2026. dirección: https://cassandra.apache.org/doc/stable/cassandra/architecture/

[17] Apache Software Foundation. «Improved Internode Messaging,» Apache Cassandra Documentation, visitado 13 de nov. de 2026. dirección: https://cassandra.apache.org/doc/stable/cassandra/architecture/messaging.html

Anexo A. Esquema inicial a mano

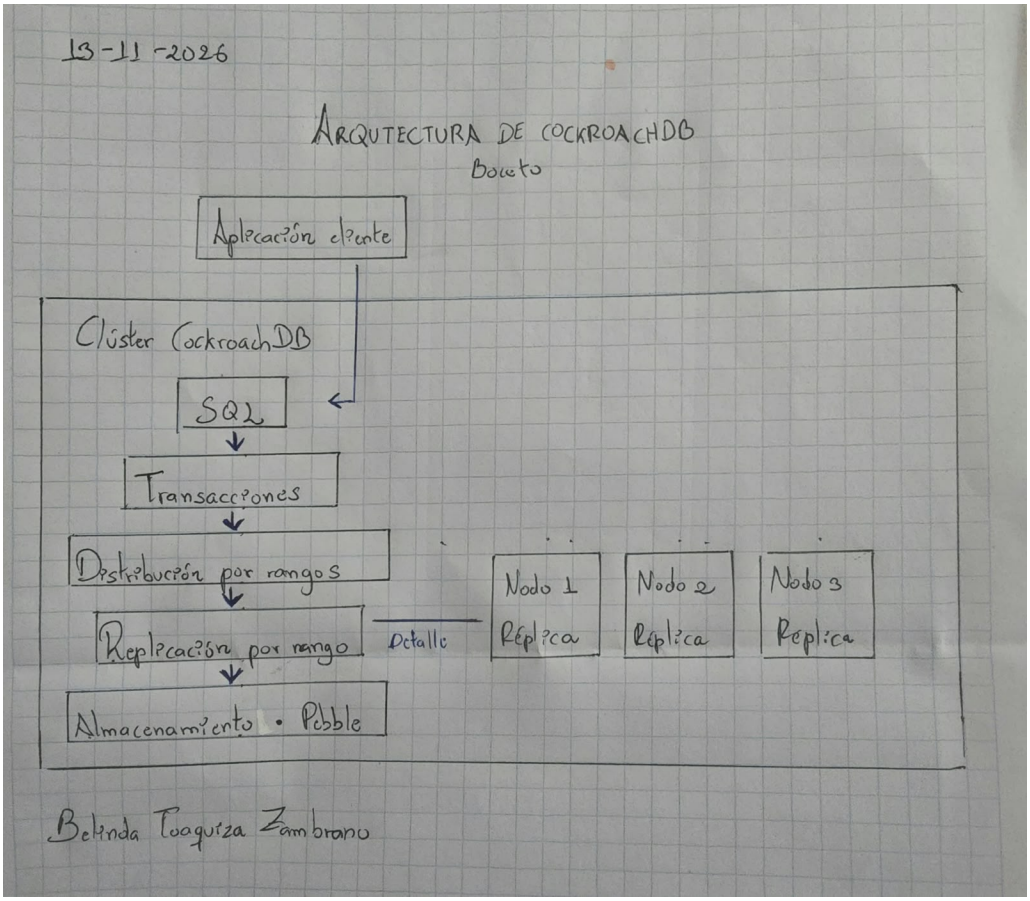
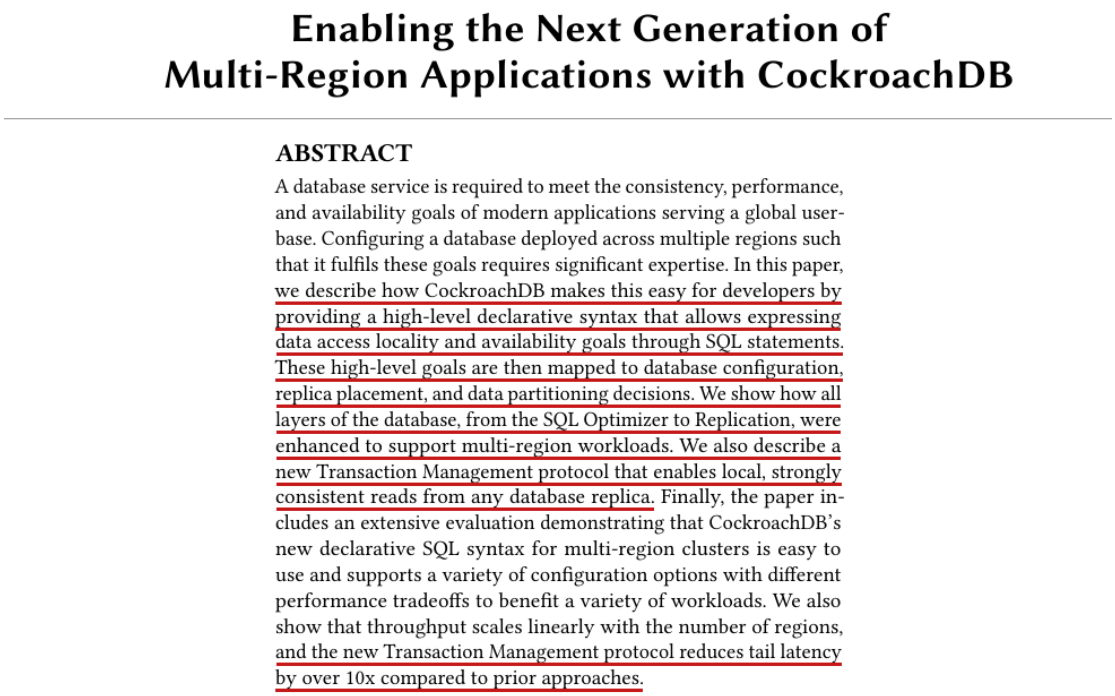


Figura 19: Esquema inicial de la arquitectura de CockroachDB, dibujado a mano.

Anexo B. Capturas de lectura de las fuentes

En cada captura se subrayó el pasaje empleado en la sección 3 y se indican el número de la referencia y la página.

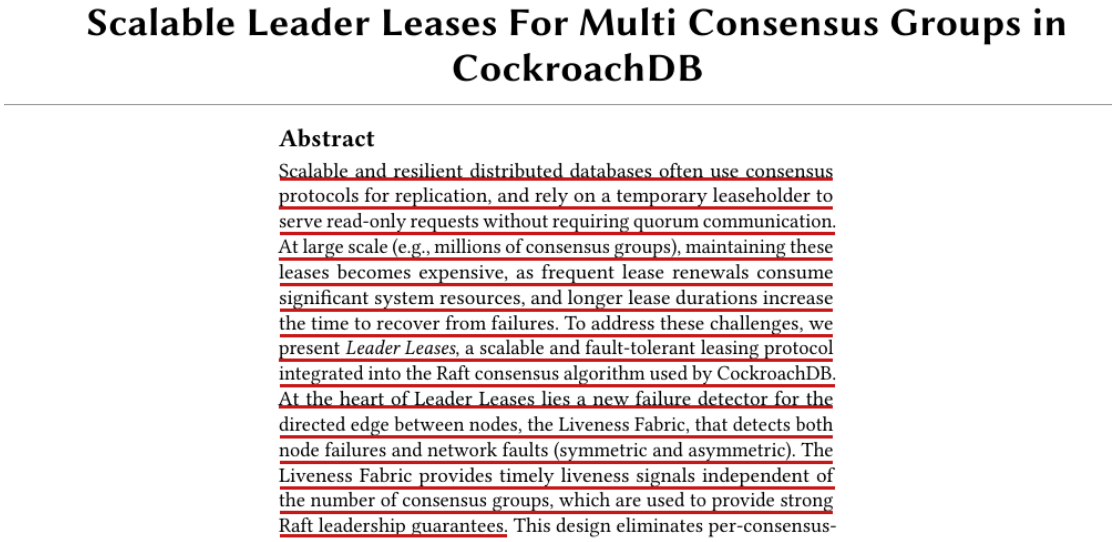


Enabling the Next Generation of
Multi-Region Applications with CockroachDB

ABSTRACT

A database service is required to meet the consistency, performance, and availability goals of modern applications serving a global user-base. Configuring a database deployed across multiple regions such that it fulfils these goals requires significant expertise. In this paper, we describe how CockroachDB makes this easy for developers by providing a high-level declarative syntax that allows expressing data access locality and availability goals through SQL statements. These high-level goals are then mapped to database configuration, replica placement, and data partitioning decisions. We show how all layers of the database, from the SQL Optimizer to Replication, were enhanced to support multi-region workloads. We also describe a new Transaction Management protocol that enables local, strongly consistent reads from any database replica. Finally, the paper includes an extensive evaluation demonstrating that CockroachDB's new declarative SQL syntax for multi-region clusters is easy to use and supports a variety of configuration options with different performance tradeoffs to benefit a variety of workloads. We also show that throughput scales linearly with the number of regions, and the new Transaction Management protocol reduces tail latency by over 10x compared to prior approaches.

Figura 20: Fuente citada [4], p. 1: se subrayó la descripción de la sintaxis declarativa que expresa objetivos de localidad y disponibilidad de los datos y la del protocolo de transacciones que habilita lecturas locales fuertemente consistentes desde cualquier réplica; ambos pasajes sostienen la afirmación de la sección 3.2 sobre configuraciones de localidad y lecturas locales en despliegues multirregión.



Scalable Leader Leases For Multi Consensus Groups in
CockroachDB

Abstract

Scalable and resilient distributed databases often use consensus protocols for replication, and rely on a temporary leaseholder to serve read-only requests without requiring quorum communication. At large scale (e.g., millions of consensus groups), maintaining these leases becomes expensive, as frequent lease renewals consume significant system resources, and longer lease durations increase the time to recover from failures. To address these challenges, we present *Leader Leases*, a scalable and fault-tolerant leasing protocol integrated into the Raft consensus algorithm used by CockroachDB. At the heart of Leader Leases lies a new failure detector for the directed edge between nodes, the Liveness Fabric, that detects both node failures and network faults (symmetric and asymmetric). The Liveness Fabric provides timely liveness signals independent of the number of consensus groups, which are used to provide strong Raft leadership guarantees. This design eliminates per-consensus-

Figura 21: Fuente citada [15], p. 1: se subrayó la presentación del mecanismo Leader Leases, su integración en el algoritmo de consenso Raft y el detector distribuido de fallos Liveness Fabric; respalda lo expuesto en las secciones 3.3 y 3.4 sobre el arrendamiento del líder y su detección de vitalidad entre nodos.

2 concurrent users. Comparatively, in its worst scenarios, Cassandra shows a 36% decrease (from around 1500 to around 970) with 2 concurrent users, and Redis demonstrates a 25% decrease (from 1389 to 1083) with 2 concurrent users.

Throughput in reading operations. Cassandra's performance is significantly lower compared to writing operations. It experiences around 65% of decrease (from 109 to 37) in the number of requests per second processed by the system in its worst-case scenario with 6 concurrent users. Meanwhile, MongoDB exhibits a 43% decrease (from 160 to 90) in its worst-case scenario with 2 users. The divergence of Cassandra's performance can be attributed to its in-memory storage, which requires runtime write validation across replicas, causing slowdowns in reads under strong consistency. In addition,

Figura 22: Fuente citada [7], p. 8: se subrayó el resultado que atribuye a Cassandra una caída del 36 % en el rendimiento de escritura y la explicación de la validación de escritura entre réplicas como causa de las lecturas lentas con consistencia fuerte; sustenta la afirmación de la sección 3.2 sobre diferencias de rendimiento asociadas a la configuración de consistencia.

The Journal of Supercomputing (2023) 79:13402–13420
https://doi.org/10.1007/s11227-023-05166-7

Check for updates

An evaluation of relational and NoSQL distributed databases on a low-power cluster

Lucas Ferreira da Silva[1] · **João V. F. Lima**[1]

Accepted: 4 March 2023 / Published online: 23 March 2023
© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2023

Abstract

The constant growth of social media, unconventional web technologies, mobile applications, and Internet of Things (IoT) devices create challenges for cloud data systems in order to support huge datasets and very high request rates. NoSQL databases, such as Cassandra and HBase, and relational SQL databases with replication, such as Citus/PostgreSQL, have been used to increase horizontal scalability and high availability of data store systems. In this paper, we evaluated three distributed databases on a low-power low-cost cluster of commodity Single-Board Computers (SBC): relational Citus/PostgreSQL and NoSQL databases Cassandra and HBase. The cluster has 15 Raspberry Pi 3 nodes with Docker Swarm orchestration tool for service deployment and ingress load balancing over SBCs. We believe that a low-cost SBC cluster can support cloud serving goals such as scale-out, elasticity, and high availability. Experimental results clearly demonstrated that there is a trade-off between performance and replication, which provides availability and partition tolerance. Besides, both properties are essential in the context of distributed systems with low-power boards. Cassandra attained better results with its consistency levels specified by the client. Both Citus and HBase enable consistency but it penalizes performance as the number of replicas increases.

Figura 23: Fuente citada [13], p. 1: se subrayó la evaluación de tres bases de datos distribuidas en un clúster de placas de bajo consumo con Docker Swarm y el resultado de Cassandra con los niveles de consistencia fijados por el cliente; respalda la mención de la sección 3.2 al clúster de placas de bajo consumo.

Software Impacts 28 (2026) 100817



Contents lists available at ScienceDirect

Software Impacts

journal homepage: www.journals.elsevier.com/software-impacts



Software update

Middleware-enforced Timed Causal Consistency for Apache Cassandra: An energy–performance–consistency evaluation against static consistency levels using YCSB



Hesam Nejati SharifAldin *, Farnoush Nayebi Pour

Department of Computer Engineering, Mashhad Branch, Islamic Azad University, Mashhad, Iran

ARTICLE INFO

Keywords:
Apache Cassandra
Consistency
Replication
Frequency-scaling
Energy–performance
Tools and monitoring

ABSTRACT

We present a reproducible and open-source middleware-enforced consistency model — Strict Timed Causal Consistency (STCC) — designed as an external consistency layer for Cassandra, systematically compared against built-in static levels (ALL, QUORUM, ONE) in terms of energy, performance, and consistency trade-offs under YCSB workloads. Cassandra's default consistency levels (e.g., ALL, QUORUM, ONE) are statically configured and unable to adapt to runtime variations in workload intensity or network conditions. To address this limitation, we introduce a fully functional middleware implementing *Strict Timed Causal Consistency* (STCC) — a hybrid model that combines client- and server-side causal guarantees with a configurable freshness bound Δ. STCC is non-intrusive and integrates seamlessly with unmodified Cassandra deployments, enabling adaptive consistency at runtime. The middleware integrates seamlessly with unmodified Cassandra instances, offering tunable freshness bounds (Δ) and causal guarantees across geo-replicated nodes without modifying the core database engine. The package includes automation tools for YCSB-based workload execution, CPU frequency control via `cpupower`, and energy monitoring using SNMP-enabled PDUs and `dstat`. This framework targets cloud engineers, distributed systems researchers, and middleware designers seeking to analyze or optimize consistency–latency–energy trade-offs under realistic deployment conditions. Our experiments validated on a 24-node Cassandra cluster distributed across three Cassandra's data centers, the system is publicly released with full documentation to support reproducible and extensible experimentation.

Figura 24: Fuente citada [14], p. 1: se subrayó la presentación de STCC como capa de consistencia externa comparada con los niveles nativos de Cassandra y la descripción de esos niveles como estáticos; sustenta la afirmación de la sección 3.2 sobre la comparación con un middleware causal externo.